

Diseño y simulación de un robot manipulador serial de seis grados de libertad para automatización logística.

Ignacio Martín Duci
ignacio8martin@gmail.com

Proyecto Final de Estudios (00441),
Facultad de Ingeniería,
Universidad Nacional de Cuyo,

Mendoza, Argentina

Mayo 2026, versión v003

Presentación parcial del Proyecto Final de Estudios para evaluación de avances

Agradecimientos

[completar]

Resumen

[completar]

Índice

1. Introducción	5
1.1. Motivación y mercado	5
1.2. Propuesta	5
2. Objetivos y beneficiarios	8
3. Descripción de la tarea y especificaciones del robot	9
4. Diseño conceptual preliminar	11
5. Modelado cinemático	13
5.1. Modelado cinemático	13
5.2. Análisis y verificación de espacio de trabajo	15
5.3. Cinemática inversa	15
5.4. Análisis de singularidades	22
6. Planificación de trayectorias	24
6.1. Generación de trayectorias	24
6.2. Escalamiento de velocidades y aceleraciones	24
7. Diseño y adaptación de CAD	26
7.1. Visualización del modelo en MATLAB	26
7.2. Diseño de piezas en <i>Fusion 360</i>	26
7.2.1. Base	27
7.2.2. Eslabón 1	27
7.2.3. Eslabón 2	28
7.2.4. Eslabón 3	28
7.2.5. Eslabón 4	28
7.2.6. Eslabón 5	29
7.2.7. Eslabón 6 – Herramienta	29
7.3. Diseño alternativo: apertura unidireccional de eslabones	30
8. Simulación y verificación de tarea	32
9. Integración en entorno ROS2	36
9.1. Arquitectura del sistema	36
9.2. Interfaces personalizadas	37
9.3. Interrelación entre nodos	39
9.4. Modelo URDF	40
9.5. Cinemática	40
9.6. Generación de trayectorias	41
9.7. Teaching	41
9.8. Interfaz gráfica	42
10. Conclusiones y trabajo futuro	46
A. Verificación y Validación de Módulos de Software de MATLAB y ROS 2	47
A.1. Cinemática Inversa	47
A.2. Singularidades	48
A.3. Generación de trayectorias	48
A.4. Integración en ROS 2	49

1. Introducción

1.1. Motivación y mercado

La automatización e informatización de procesos de servicios es una trayectoria que, con el pasar de los años, aumenta su presencia en los diferentes mercados, brindando un aumento de la eficiencia, precisión, seguridad, operatividad y trazabilidad. Estos beneficios que brinda la implementación de tecnologías también quitan competitividad a aquellas entidades que, por disponibilidad, costo o falta de personal capacitado, no pueden implementar.

Al revisar datos aportados por la consultora *Grand View Research*, para analizar las proyecciones de mercado 2023-2030 para la robótica aplicada a depósitos logísticos o su traducción al inglés *warehouses* para el mundo [1], Estados Unidos [2], China [3], Latinoamérica [4] y Brasil [5] pueden elaborarse las siguientes conclusiones.

1. Todos los mercados proyectan un crecimiento anual compuesto de entre 17 % y 22.1 % lo cual se considera por encima de un escenario favorable.
2. Comparativamente, Latinoamérica proyecta un escenario de crecimiento mayor al de Estados Unidos, China y al mundo en totalidad.
3. Al analizar un país limítrofe que reporte datos como Brasil, puede verse que proyecta un crecimiento similar al de China en el período de tiempo analizado.

Bajo estas conclusiones, puede entenderse que el mercado continuará creciendo, y que países con menores niveles de automatización e informatización serán los que presentarán mejores oportunidades para este campo. Si se considera la situación actual de Argentina, orientada a un modelo productivo con menor carga impositiva, puede posicionarse como plataforma estratégica para fabricar y exportar robótica de servicio.

En el contexto de la automatización de los procesos logísticos, la implementación de robots manipuladores es esencial para lograr movimientos en el espacio con alto grado de flexibilidad que permitan la reorganización de paquetes con alta precisión y eficiencia.

El desarrollo desde cero de un robot tipo serie de seis grados de libertad presenta una excelente oportunidad para integrar los conocimientos adquiridos en la carrera. También se presentarán desafíos que requerirán expandir conocimientos y aplicar nuevos conceptos y técnicas con criterio. La robótica serie es un campo profundamente estudiado, por lo que podrá accederse a una gran cantidad de información ya existente.

1.2. Propuesta

En el presente trabajo se desarrolla el diseño y la simulación de un robot manipulador serial de seis grados de libertad orientado a una aplicación específica: la transferencia de objetos entre contenedores (*bin-to-bin transfer*) de hasta 3 kg.

Esta aplicación se plantea como un caso representativo dentro de procesos de manipulación en depósitos logísticos automatizados (*micro-fulfillment warehouses*), permitiendo validar el desempeño del sistema en tareas típicas de pick-and-place.

La selección del caso de estudio se inspira en desarrollos industriales existentes, como el presentado en [YouTube](#). No obstante, la solución propuesta en este trabajo es de desarrollo propio, enfocada en el modelado cinemático, la generación de trayectorias y la simulación del comportamiento del manipulador.

En la figura 1 puede verse un esquema de la aplicación descrita.

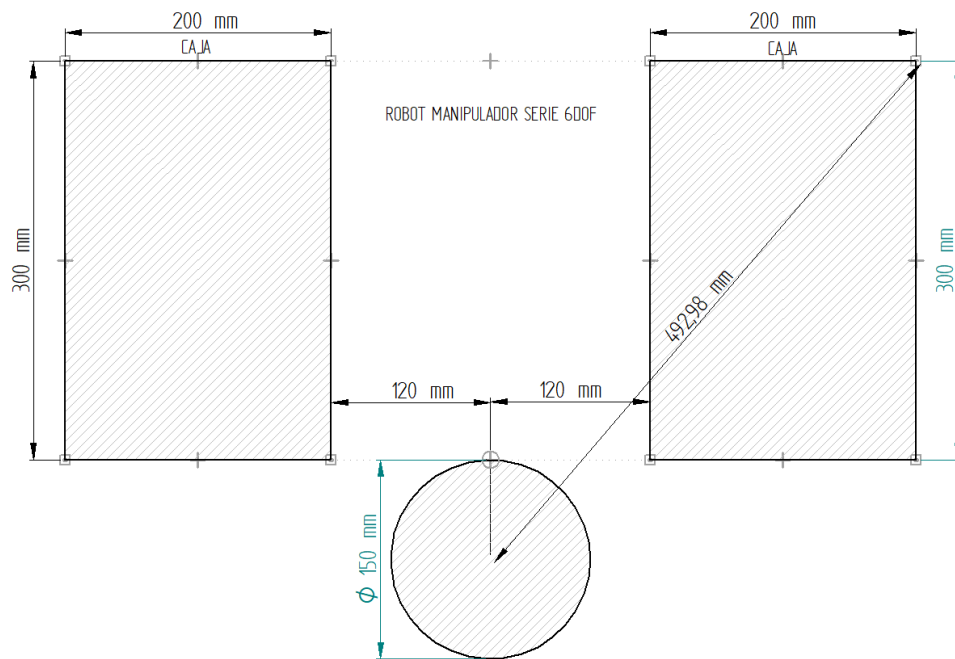


Figura 1: Esquema de la aplicación - vista superior.

Mínimamente, se propone que el robot tenga un alcance de 500 mm con una capacidad de carga útil de hasta 3 kg, similar a las características del robot comercial [UR3e](#). En la figura 2 se puede ver un esquema general de la morfología del robot.

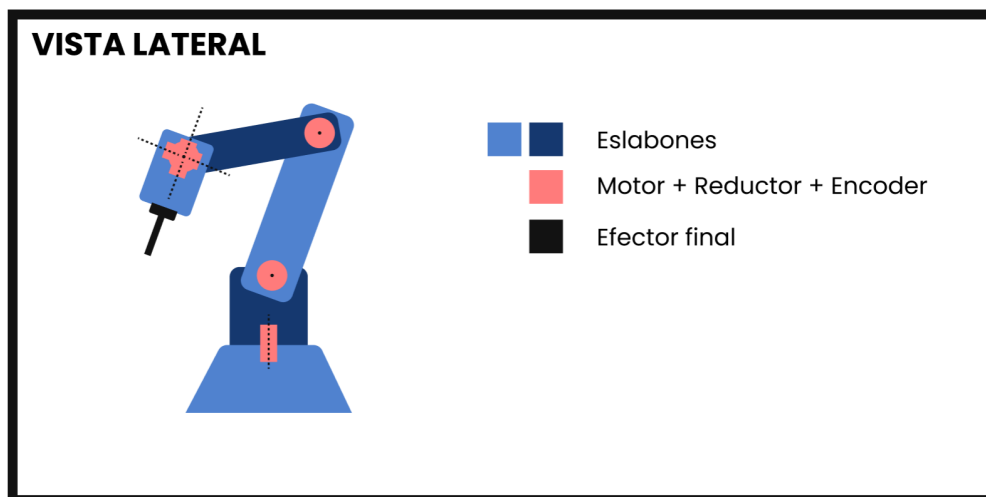


Figura 2: Esquema del robot tipo serie de 6 GDL propuesto.

La tarea del robot consiste cronológicamente en lo siguiente:

1. Roto-traslación a postura de espera de forma rápida.
2. Roto-traslación a postura sobre caja origen a alta velocidad.
3. Roto-traslación para *picking* a baja velocidad.
4. Roto-traslación hacia postura sobre caja origen a baja velocidad.
5. Roto-traslación hacia postura sobre caja destino a alta velocidad.
6. Roto-traslación para *placing* a baja velocidad.

7. Roto-traslación hacia postura sobre caja destino a baja velocidad.

8. Repetir.

Dentro de la bibliografía a consultar inicialmente se mencionan los libros [7], [8], [9], [10], [11], [12] y [13].

2. Objetivos y beneficiarios

Objetivo mínimo: Diseñar y simular un robot manipulador de seis ejes tipo serie con 500 mm de alcance y 3 kg de carga útil destinado a tareas de *bin-to-bin transfer* capaz de mover 300 elementos por hora.

Los siguientes objetivos se desarrollan en dos etapas: los ítems 1 a 6 corresponden a la implementación y validación en entorno MATLAB, mientras que los ítems subsiguientes se desarrollan en el entorno ROS 2.

1. Definir las especificaciones del robot y su tarea con un primer acercamiento hacia las normas de seguridad de máquinas, robótica y colaboración humano-máquina.
2. Realizar un diseño conceptual preliminar.
3. Analizar y validar el espacio de trabajo.
4. Resolver analíticamente y validar la cinemática inversa.
5. Implementar y validar un programa de generación de trayectorias que incorpore distintas funcionalidades, permitiendo la selección de modos de interpolación (cartesiana y articular) y la gestión de singularidades.
6. Implementar, simular y validar la ejecución de la tarea definida, asegurando su correcto desempeño y cumplimiento de los objetivos planteados.
7. Desarrollar un modelo CAD para simulaciones cinemáticas, asegurando la correspondencia entre los sistemas de referencia de cada componente y los marcos definidos mediante la convención Denavit-Hartenberg, a fin de garantizar consistencia geométrica y fidelidad en la representación del sistema en entornos de simulación.
8. Desarrollar el modelo del robot en formato URDF a partir de la parametrización cinemática definida (DH), asegurando la correcta correspondencia entre eslabones y articulaciones.
9. Integrar y verificar la coherencia de las geometrías CAD como representaciones visuales del modelo.
10. Implementar el desarrollo de cinemática inversa, cinemática directa y generación de trayectorias en ROS 2.
11. Desarrollar nodos de *teaching* y gestión de trayectorias en ROS 2.
12. Desarrollar nodo de interfaz de usuario gráfica que permita el movimiento manual, enseñanza y gestión de trayectorias del robot, así como análisis de variables cinemáticas en ROS 2.
13. Implementar formato de descripción universal del robot (URDF) para simular con mayor estética el robot en entorno RViz.

Posibles beneficiarios:

1. **Bodegas y agro-industria mendocina** que requieren automatizar el empaque y la manipulación de insumos sin invertir en robots industriales de alto costo.
2. **Centros de *micro-fulfillment* y PYMES logísticas** donde el brazo puede aumentar el *pick-rate* y reducir tiempos de entrega.
3. **Institutos y universidades locales** (UNCuyo, ITU, UTN) que obtendrán una plataforma didáctica *open-source* para prácticas de mecatrónica y control.

3. Descripción de la tarea y especificaciones del robot

La tarea del manipulador consiste en la transferencia de objetos entre dos contenedores de dimensiones y posiciones predefinidas. Esto puede verse en las figuras 3 y 4 donde también quedan definidos puntos clave de la trayectoria que deberá seguir el robot conforme a lo definido en la sección 1.2.

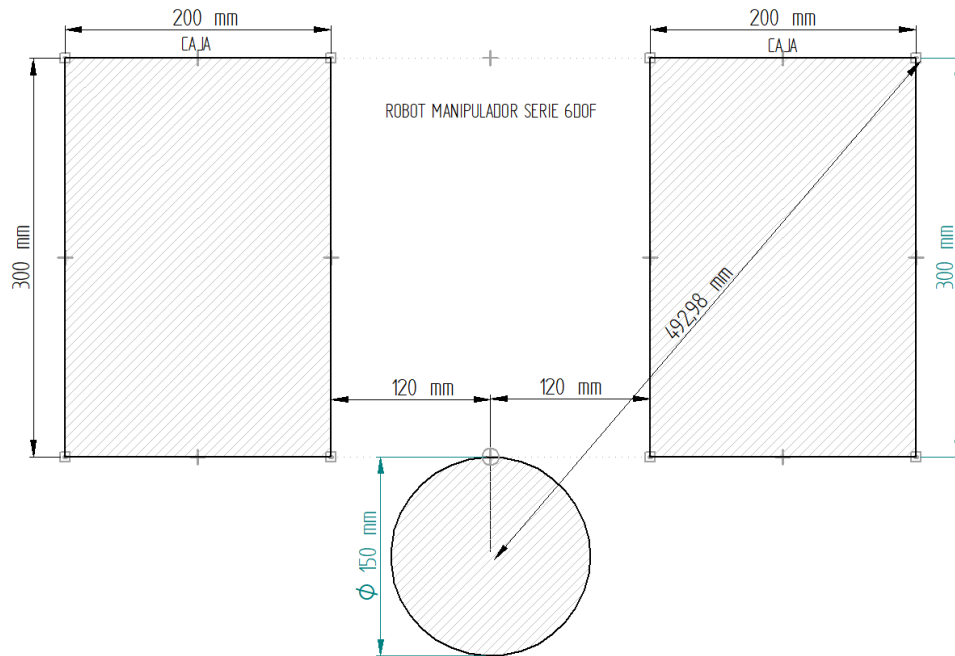


Figura 3: Vista superior del robot y la tarea

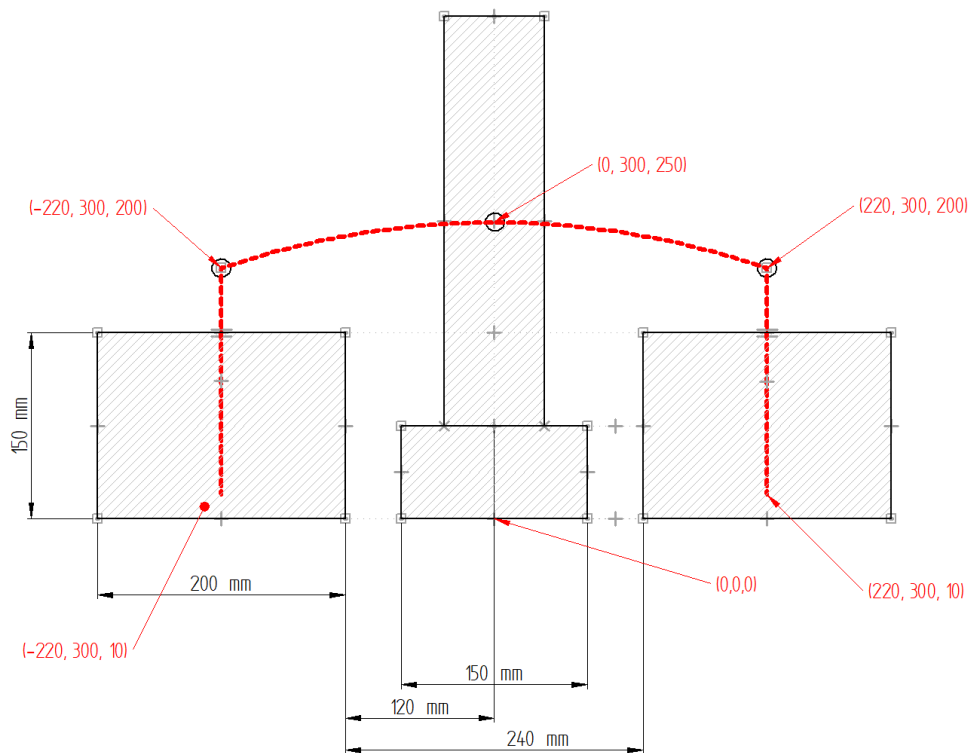


Figura 4: Vista lateral del robot y la tarea

Los objetos serán tomados mediante un sistema de sujeción por vacío (ventosa - *vacuum gripper*) y depositados mediante la liberación del mismo, esto permite relajar los requerimientos de precisión y repetibilidad del manipulador.

Quedan definidas las especificaciones principales del manipulador en función de la tarea descrita¹:

- Alcance: 500.00 mm
- Capacidad de carga útil: 3.00 kg
- Precisión: 0.5 mm
- Repetibilidad: 0.5 mm
- Grados de libertad (GDL - *DOF*): 6.
- Rango articulaciones $1:5 \pm 2\pi$
- Rango articulación final *libre*

Se definen también otras especificaciones y condiciones del entorno, posiblemente útiles para desarrollos posteriores:

- Temperatura ambiente entre 5 y 35 °C.
- Humedad relativa $\leq 70\%$.
- Bajo nivel de polvo ambiental.
- Iluminación 100-150 lx.
- Entorno sin vibraciones.
- Protección IP54

¹Se considera un alcance y una capacidad de carga superiores a los requeridos por la tarea, con el fin de incorporar un margen de diseño que garantice un funcionamiento adecuado del sistema y permita su adaptación a tareas similares.

4. Diseño conceptual preliminar

Con el objetivo de tener un punto de partida en cuanto a la morfología y dimensiones del robot, tomando como referencia al robot [UR3e](#), se define la siguiente estructura, morfología y dimensiones:

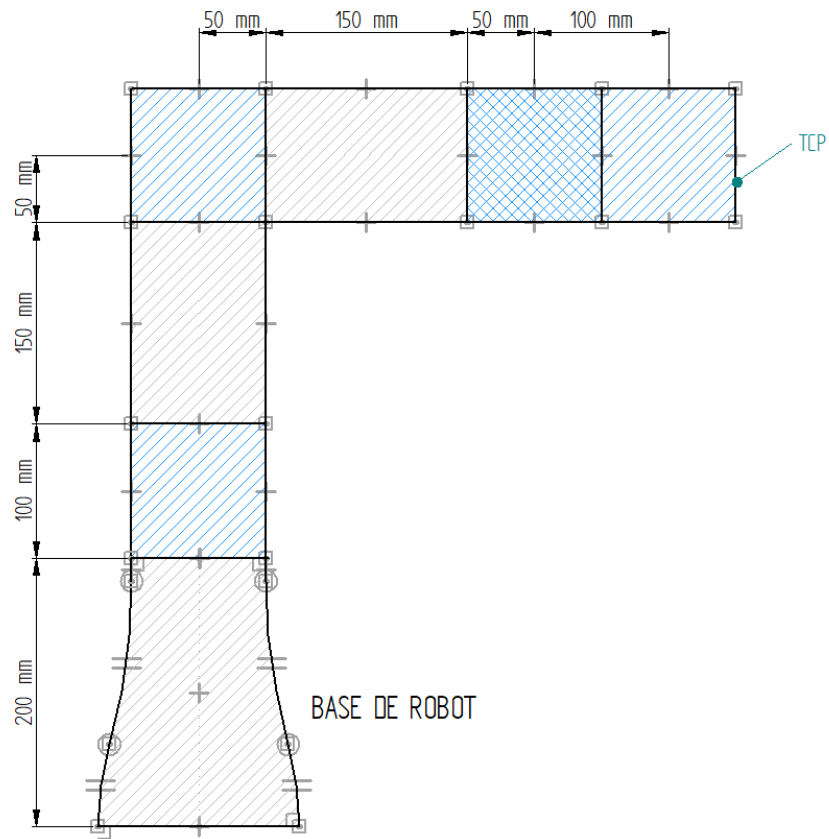


Figura 5: Vista lateral 1 de un esquema del robot

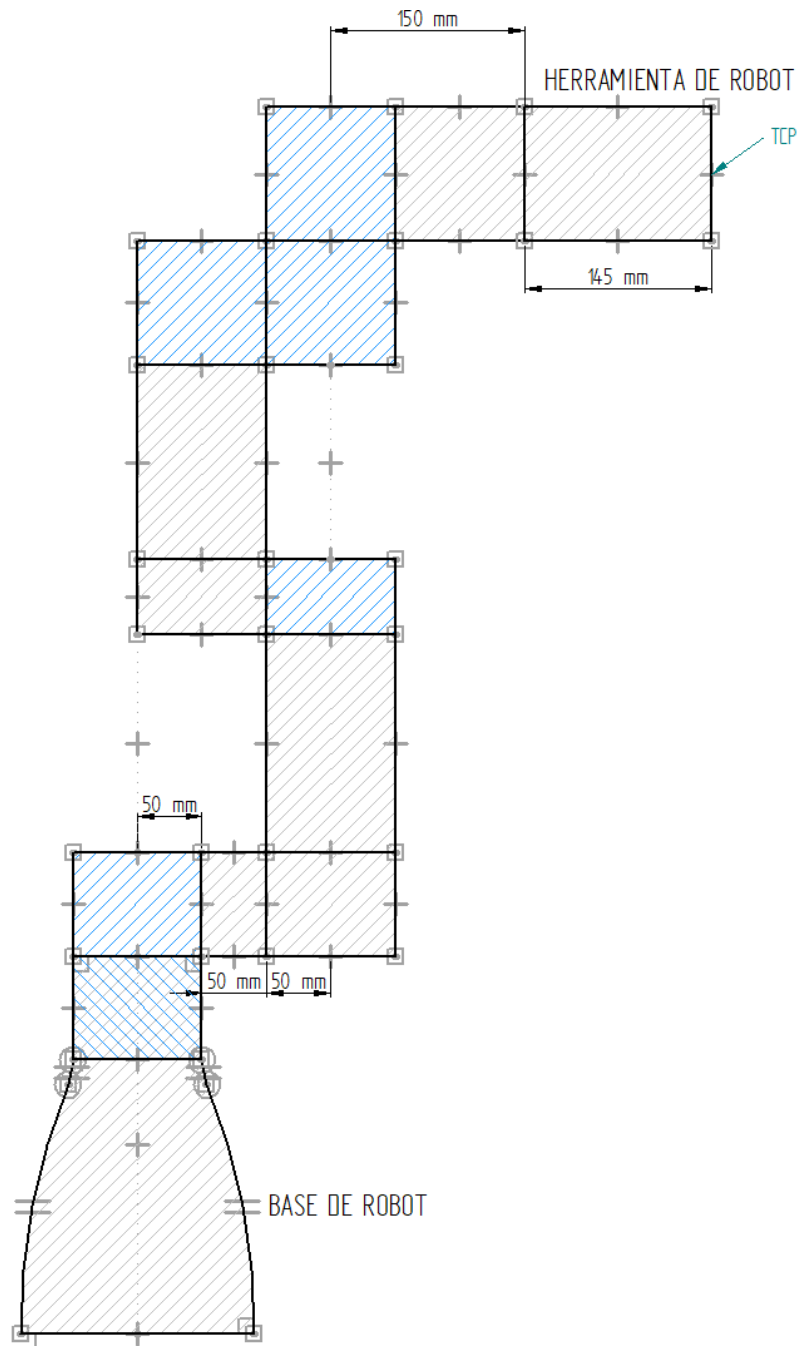


Figura 6: Vista lateral 2 del un esquema del robot

A partir de estas imágenes se obtiene la información necesaria para la elaboración del modelo cinemático. Se ha incorporado una base para permitir que el robot opere de forma segura por encima de las cajas y una herramienta final tipo ventosa (*vacuum-gripper*), para esta última se toma como referencia el modelo *EPick with one suction cup 145x83 mm* del fabricante [Robotiq](#).

5. Modelado cinemático

En esta sección se aborda el modelado cinemático del manipulador, así como la resolución de la cinemática inversa y la planificación de trayectorias necesarias para su operación.

El modelado cinemático permite establecer la relación entre las variables articulares del robot y la posición y orientación de su efector final. A partir de este modelo, se plantea la resolución de la cinemática inversa, con el objetivo de determinar las configuraciones articulares requeridas para alcanzar posiciones específicas dentro del espacio de trabajo.

Finalmente, se desarrolla la planificación de trayectorias, mediante la cual se define la evolución temporal del movimiento del manipulador entre distintas configuraciones, garantizando continuidad, suavidad y seguridad en su ejecución.

5.1. Modelado cinemático

Para obtener el modelo cinemático se aplica el algoritmo Denavit-Hartenberg estándar el cual consiste en lo siguiente:

1. Enumerar los $n+1$ eslabones desde 0 hasta n , comenzando desde la base (eslabón fijo) y terminando en el efector final.
2. Identificar los ejes de cada articulación. Si es rotacional, será el eje de giro; si es prismática, será el eje a lo largo del cual se produce el desplazamiento.
3. Enumerar los ejes de 1 a n , comenzando desde el que une el eslabón base con el eslabón 1.
4. Para i de 0 a $n-1$, situar el eje Z_1 en el eje de la articulación $i+1$.
5. El eje Z_n se colocará en el extremo del último eslabón en la misma dirección que el eje Z_{n-1} .
6. Situar el origen del sistema de la base $\{S_0\}$ en cualquier punto del eje Z_0 .
7. Para $i = 1$ a n , ubicar el sistema $\{S_i\}$ en la intersección entre el eje Z_i y la recta perpendicular común a los ejes Z_i y Z_{i-1} . En caso de que los ejes Z_i y Z_{i-1} se intersecten, el sistema $\{S_i\}$ se posiciona en dicho punto de intersección.
8. Para $i = 1$ a n , definir el eje X_i a partir del punto donde se estableció el sistema $\{S_i\}$, sobre la recta perpendicular común a los ejes Z_i y Z_{i-1} . Si los ejes Z_i y Z_{i-1} se intersectan, el eje X_i debe ser perpendicular a ambos. Su orientación puede elegirse de forma arbitraria.
9. El eje X_0 puede definirse libremente, siendo conveniente alinearlos con el eje X_1 .
10. Para $i = 0$ a n , definir el eje Y_i de manera que, junto con los ejes X_i y Z_i , forme un sistema de referencia dextrógiro.

Luego se determinan los parámetros que permitirán elaborar una matriz que define al robot en cuestión de la siguiente forma:

Para $i = 1$ a n :

1. θ_i : Ángulo alrededor del eje Z_{i-1} , desde el eje X_{i-1} hasta el eje X_i .
2. d_i : Distancia a lo largo del eje Z_{i-1} , desde el origen del sistema $i - 1$ hasta el eje X_i .
3. a_i : Distancia a lo largo del eje X_i , desde el eje Z_{i-1} hasta el eje Z_i .
4. α_i : Ángulo alrededor del eje X_i , desde el eje Z_{i-1} hasta el eje Z_i .

Donde:

- a_i : Longitud del eslabón.
- α_i : Ángulo de torsión del eslabón.
- d_i : Longitud articular.

- θ_i : Ángulo articular.

Utilizando esta información, los parámetros de Denavit-Hartenberg pueden organizarse en la siguiente forma general:

i	θ_i	d_i	a_i	α_i
1	θ_1	d_1	a_1	α_1
2	θ_2	d_2	a_2	α_2
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	θ_n	d_n	a_n	α_n

(1)

Finalmente, se ha conseguido describir la geometría del robot y la interrelación entre sus eslabones mediante un algoritmo. El beneficio de esta representación radica en que permite determinar la posición y orientación de cada sistema de referencia a partir del anterior y, por consiguiente, obtener la posición y orientación del efector final en función de las variables articulares. A continuación, se presenta de forma general la matriz de transformación homogénea que relaciona cada sistema de referencia con el anterior.

$${}^{i-1}T_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2)$$

Puede entonces determinarse, para un robot de 6 grados de libertad, la posición y orientación del efector final en función de las variables articulares mediante la transformación sucesiva del sistema base hasta el sistema herramienta.

$${}^{base}T_{tcp} = {}^bT_0 {}^0T_1 \dots {}^5T_6 {}^6T_{tcp} \quad (3)$$

De esta forma quedará definida **el modelo cinemático directo**.

Se obtiene información sobre las dimensiones del robot de la figura 5 y 6 y se elabora el siguiente esquema simplificado con información solo esencial al aplicar el algoritmo DH:

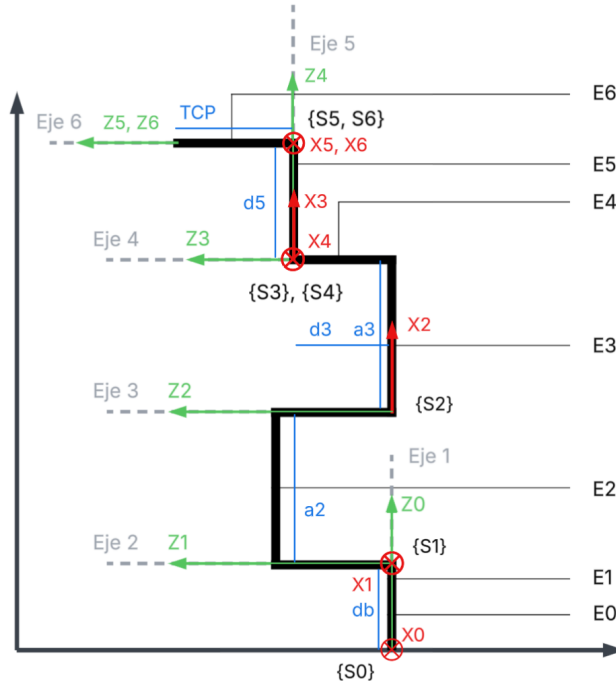


Figura 7: Algoritmo DH aplicado a un esquema simplificado del robot

Teniendo en cuenta que las articulaciones son de tipo rotacional, la variable articular será θ para todos los casos. Se define la matriz Denavit-Hartenberg a partir de la información de la figura 7 y del procedimiento definido en la explicación del algoritmo:

i	$\theta_i[rad]$	$d_i[m]$	$a_i[m]$	$\alpha_i[rad]$
1	q_1	0,25	0	$-\frac{\pi}{2}$
2	$q_2 - \frac{\pi}{2}$	0	0,25	0
3	q_3	0,295	0,25	0
4	$q_4 + \frac{\pi}{2}$	0	0	$\frac{\pi}{2}$
5	q_5	0,1	0	$-\frac{\pi}{2}$
6	q_6	0	0	0

(4)

Notas:

- Mediante la letra "q" se simboliza el parámetro variable.
- La suma o resta a valores de q , como lo es en los casos 2 y 4, dan un offset de posición que modifica el punto cero del robot.

5.2. Análisis y verificación de espacio de trabajo

Para verificar el alcance máximo y la tarea de nuestro robot, simplemente buscamos una posición de alto estiramiento ($q = [variable, \frac{\pi}{2}, 0, -\frac{\pi}{2}, -\frac{\pi}{2}, 0]$) y se ingresan valores progresivos completando el giro de la primera articulación. Luego, esto produce el siguiente resultado (script `test_workspace.m`):

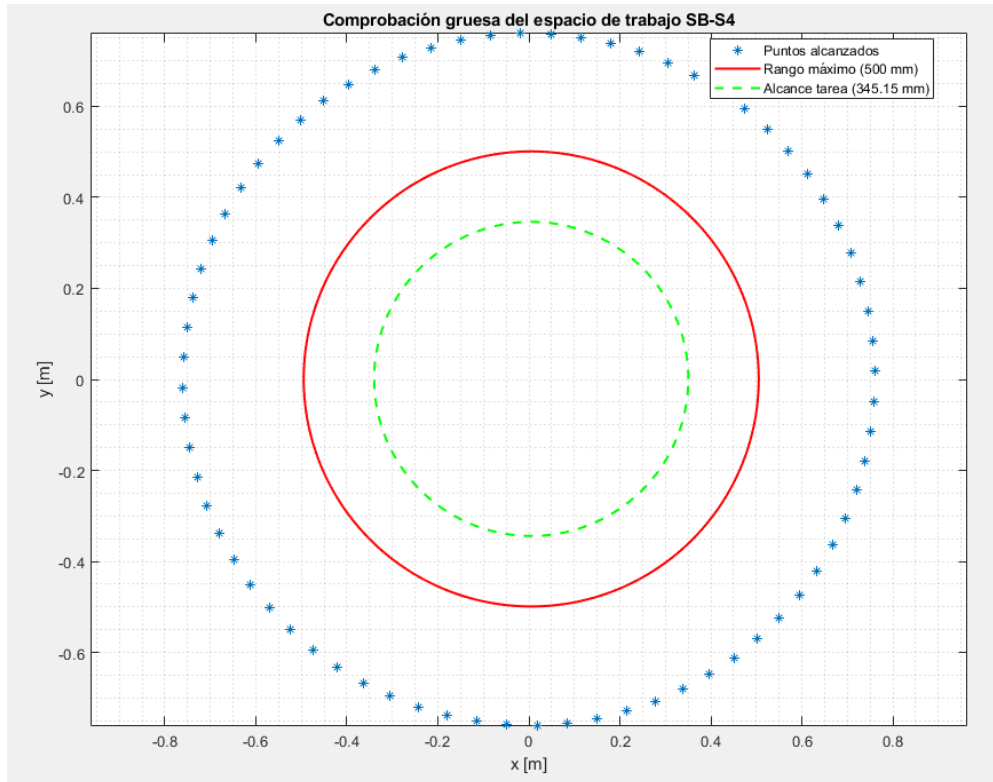


Figura 8: Verificación de espacio de trabajo en plano XY

5.3. Cinemática inversa

En el desarrollo cinemático de todo robot es fundamental elaborar el modelo de cinemática inversa, el cual permite determinar las variables articulares necesarias para alcanzar una posición y orientación

deseadas del efector final. En este trabajo, la resolución se aborda mediante un enfoque analítico, obteniendo expresiones en forma cerrada que relacionan directamente las variables del espacio cartesiano con las variables articulares, evitando el uso de métodos numéricos iterativos.

Se parte de la observación de que el manipulador no cuenta con una muñeca esférica; es decir, los tres ejes correspondientes a la muñeca no se intersectan en un mismo punto, lo que no nos permite hacer uso del método de Pieper.

En esta resolución, se implementan métodos algebraicos para separar incógnitas y determinarlas en función de parámetros conocidos y variables ya determinadas.

Dado que se ha definido el modelo cinemático directo al haber elaborado la matriz Denavit-Hartenberg se conocen todas las matrices de transformación homogénea, particularmente:

$$T_{tcp}^{base}, T_0^{base}, T_{tcp}^6 \quad (5)$$

Al desacoplar la base y la herramienta final se puede conocer:

$$T_6^0 = [T_0^{base}]^{-1} T_{tcp}^{base} [T_{tcp}^6]^{-1} \quad (6)$$

También se sabe que:

$$T_6^0 = T_1^0 T_2^1 T_3^2 T_4^3 T_5^4 T_6^5 \quad (7)$$

En esta ecuación hay una alta complejidad en el lado derecho (RHS) y una baja complejidad en el lado izquierdo (LSH) por lo que se equilibra de la siguiente manera:

$$[T_1^0]^{-1} T_6^0 [T_6^5]^{-1} = T_2^1 T_3^2 T_4^3 T_5^4 \quad (8)$$

De forma genérica la matriz de transformación homogénea que vincula al sistema $i - 1$ con el sistema i es:

$$T_i^{i-1} = \begin{bmatrix} c_i & -s_i c_j & s_i s_j & a_i c_i \\ s_i & c_i c_j & -s_i s_j & a_i s_i \\ 0 & s_j & c_j & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (9)$$

Donde:

$$\begin{aligned} s_i &:= \sin(\theta_i) \\ c_i &:= \cos(\theta_i) \\ s_j &:= \sin(\alpha_i) \\ c_j &:= \cos(\alpha_i) \end{aligned} \quad (10)$$

NOTA: El subíndice i y j no indican sistemas diferentes sino que i hace referencia al ángulo θ y j al ángulo α para simplificar la nomenclatura de las funciones.

Cada matriz de transformación homogénea esta compuesta por cuatro partes:

$$[T_i^{i-1}] = \begin{bmatrix} R_{3 \times 3} & p_{3 \times 1} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Se puede obtener la matriz de transformación homogénea inversa mediante la siguiente expresión:

$$\begin{aligned} [T_i^{i-1}]^{-1} &= \begin{bmatrix} R^T & -R^T T \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ R^T &= \begin{bmatrix} c_i & s_i & 0 \\ -s_i c_j & c_i c_j & s_j \\ s_i s_j & -c_i s_j & c_j \end{bmatrix} \\ p &= \begin{bmatrix} a_i c_i \\ a_i s_i \\ d_i \end{bmatrix} \end{aligned}$$

$$-R^T p = \begin{bmatrix} -c_i & -s_i & 0 \\ +s_i c_j & -c_i c_j & -s_j \\ -s_i s_j & +c_i s_j & -c_j \end{bmatrix} \begin{bmatrix} a_i c_i \\ a_i s_i \\ d_i \end{bmatrix} = \begin{bmatrix} -a_i \\ -d_i s_j \\ -d_i c_j \end{bmatrix}$$

Y se obtiene la matriz de transformación homogénea inversa:

$$[T_i^{i-1}]^{-1} = \begin{bmatrix} c_i & s_i & 0 & -a_i \\ -s_i c_j & c_i c_j & s_j & -d_i s_j \\ s_i s_j & -c_i s_j & c_j & -d_i c_j \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Se toma esta plantilla de matriz y la plantilla de la matriz 9, se reemplazan valores y se simplifican para formar las matrices de transformación homogéneas presentes en la ecuación 8.

$$[T_1^0]^{-1} = \begin{bmatrix} c_1 & s_1 & 0 & 0 \\ 0 & 0 & -1 & d_1 \\ -s_1 & c_1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_6^5]^{-1} = \begin{bmatrix} c_6 & s_6 & 0 & 0 \\ -s_6 & c_6 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_2^1] = \begin{bmatrix} s_2 & c_2 & 0 & a_2 s_2 \\ -c_2 & s_2 & 0 & -a_2 c_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_3^2] = \begin{bmatrix} c_3 & -s_3 & 0 & a_3 c_3 \\ s_3 & c_3 & 0 & a_3 s_3 \\ 0 & 0 & 1 & d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_4^3] = \begin{bmatrix} -s_4 & 0 & c_4 & 0 \\ c_4 & 0 & s_4 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_5^4] = \begin{bmatrix} c_5 & 0 & -s_5 & 0 \\ s_5 & 0 & c_5 & 0 \\ 0 & -1 & 0 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_6^0] = \begin{bmatrix} r_{11} & r_{12} & r_{13} & x \\ r_{21} & r_{22} & r_{23} & y \\ r_{31} & r_{32} & r_{33} & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

La ecuación 8, con sus matrices ya determinadas, se carga a un script de MATLAB `sym_1.m` para, mediante trabajo simbólico, determinar la expresión final simplificada de cada lado de la ecuación; se obtiene:

$$\text{LHS} = \begin{bmatrix} c_6(r_{11}c_1 + r_{21}s_1) - s_6(r_{12}c_1 + r_{22}s_1) & c_6(r_{12}c_1 + r_{22}s_1) + s_6(r_{11}c_1 + r_{21}s_1) & r_{13}c_1 + r_{23}s_1 & xc_1 + ys_1 \\ r_{32}s_6 - r_{31}c_6 & -r_{32}c_6 - r_{31}s_6 & -r_{33} & d_1 - z \\ c_6(r_{21}c_1 - r_{11}s_1) - s_6(r_{22}c_1 - r_{12}s_1) & c_6(r_{22}c_1 - r_{12}s_1) + s_6(r_{21}c_1 - r_{11}s_1) & r_{23}c_1 - r_{13}s_1 & yc_1 - xs_1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\text{RHS} = \begin{bmatrix} c_5 c_{234} & -s_{234} & -s_5 c_{234} & a_3 s_{23} + a_2 s_2 + d_5 s_{234} \\ c_5 s_{234} & c_{234} & -s_5 s_{234} & -a_3 c_{23} - a_2 c_2 - d_5 c_{234} \\ s_5 & 0 & c_5 & d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Es importante resaltar que los valores r_{ij} , x , y y z , son valores conocidos; son la información de entrada al cálculo de la cinemática inversa. También es necesario aclarar que $c_{abc} = \cos(a + b + c)$ y $s_{abc} = \sin(a + b + c)$.

Las ecuaciones formadas por los elementos de las matrices LHS y RHS son los siguientes:

$$c_6(r_{11}c_1 + r_{21}s_1) - s_6(r_{12}c_1 + r_{22}s_1) = c_5 c_{234} \quad (11)$$

$$c_6(r_{12}c_1 + r_{22}s_1) + s_6(r_{11}c_1 + r_{21}s_1) = -s_{234} \quad (12)$$

$$r_{13}c_1 + r_{23}s_1 = -s_5 c_{234} \quad (13)$$

$$xc_1 + ys_1 = a_3 s_{23} + a_2 s_2 + d_5 s_{234} \quad (14)$$

$$r_{32}s_6 - r_{31}c_6 = c_5 s_{234} \quad (15)$$

$$-r_{32}c_6 - r_{31}s_6 = c_{234} \quad (16)$$

$$-r_{33} = -s_5 s_{234} \quad (17)$$

$$d_1 - z = -a_3 c_{23} - a_2 c_2 - d_5 c_{234} \quad (18)$$

$$c_6(r_{21}c_1 - r_{11}s_1) - s_6(r_{22}c_1 - r_{12}s_1) = s_5 \quad (19)$$

$$c_6(r_{22}c_1 - r_{12}s_1) + s_6(r_{21}c_1 - r_{11}s_1) = 0 \quad (20)$$

$$r_{23}c_1 - r_{13}s_1 = c_5 \quad (21)$$

$$yc_1 - xs_1 = d_3 \quad (22)$$

Se toma la ecuación formada por la posición 3:4 (fila 3, columna 4, ecuación 22):

$$yc_1 - xs_1 = d_3 \quad (23)$$

$$yc_1 - xs_1 - d_3 = 0 \quad (24)$$

$$A = y, B = -x, C = -d_3 \quad (25)$$

$$Ac_1 + Bs_1 + C = 0 \quad (26)$$

Se aplica la técnica de sustitución por la tangente de medio ángulo:

$$\begin{aligned} t &= \tan \frac{\theta_1}{2} \\ \cos(\theta_1) &= \frac{1 - t^2}{1 + t^2} \\ \sin(\theta_1) &= \frac{2t}{1 + t^2} \end{aligned} \quad (27)$$

$$A \frac{1-t^2}{1+t^2} + B \frac{2t}{1+t^2} + C = 0 \quad (28)$$

$$A(1-t^2) + B2t + C(1+t^2) = 0 \quad (29)$$

$$A - At^2 + B2t + C + Ct^2 = 0 \quad (30)$$

$$t^2(C - A) + t(2B) + (C + A) = 0 \quad (31)$$

$$t_{1,2} = \frac{-2B \pm \sqrt{4B^2 - 4(C - A)(C + A)}}{2(C - A)} \quad (32)$$

De esta última ecuación se pueden determinar dos valores que conducen a dos valores de θ_1 mediante:

$$\theta_{1-1,2} = 2 \arctan(t_{1,2}) \quad (33)$$

La sustitución de medio ángulo $t = \tan(\theta/2)$ establece una relación biyectiva para $\theta \in (-\pi, \pi)$. De este modo, al recuperar el ángulo mediante $\theta = 2 \tan^{-1}(t)$, se extiende el rango de la función arco tangente y se evita la ambigüedad de cuadrante.

Una vez determinados ambos valores para q_1 , se analiza la ecuación formada por los miembros de la posición 3:2 (fila 3, columna 2, ecuación 20):

$$c_6(r_{22}c_1 - r_{12}s_1) + s_6(r_{21}c_1 - r_{11}s_1) = 0 \quad (34)$$

$$A = -(r_{22}c_1 - r_{12}s_1) \quad (35)$$

$$B = r_{21}c_1 - r_{11}s_1$$

$$Ac_6 + Bs_6 = 0 \quad (36)$$

$$\frac{s_6}{c_6} = t_6 \quad (37)$$

$$\theta_6 = \text{atan2}(t_6) \quad (38)$$

De esta última ecuación se obtiene un θ_6 por cada valor de θ_1 .

Se toma la ecuación 3:1 (fila 3, columna 1, ecuación 19) y la ecuación 3:3 (fila 3, columna 3, ecuación 21):

$$s_5 = c_6(r_{21}c_1 - r_{11}s_1) - s_6(r_{22}c_1 - r_{12}s_1) \quad (39)$$

$$c_5 = r_{23}c_1 - r_{13}s_1$$

$$\frac{s_5}{c_5} = t_5 \quad (40)$$

$$\theta_5 = \text{atan2}(t_5) \quad (41)$$

Para determinar correctamente los valores de θ_5 es fundamental mantener trazabilidad entre los resultados obtenidos para las variables articulares que intervienen. De esta manera se obtiene un valor de θ_5 por cada combinación de θ_1 y θ_6 .

Desde las ecuaciones formadas en las posiciones 1:3 (fila 1, columna 3, ecuación 13) y 2:3 (fila 2, columna 3, ecuación 17):

$$\begin{aligned} -s_5c_{234} &= r_{13}c_1 + r_{23}s_1 \\ -s_5s_{234} &= -r_{33} \end{aligned} \quad (42)$$

$$\begin{aligned} A = c_{234} &= \frac{r_{13}c_1 + r_{23}s_1}{-s_5} \\ B = s_{234} &= \frac{r_{33}}{s_5} \end{aligned} \quad (43)$$

Se continúa tomando las ecuaciones de las posiciones 1:4 (fila 1, columna 4, ecuación 14) y 2:4 (fila 2, columna 4, ecuación 18):

$$\begin{aligned} xc_1 + ys_1 &= a_3s_{23} + a_2s_2 + d_5s_{234} \\ d_1 - z &= -a_3c_{23} - a_2c_2 - d_5c_{234} \end{aligned} \quad (44)$$

Se reemplaza lo obtenido previamente en estas ecuaciones:

$$\begin{aligned} xc_1 + ys_1 &= a_3s_{23} + a_2s_2 + d_5B \\ d_1 - z &= -a_3c_{23} - a_2c_2 - d_5A \end{aligned} \quad (45)$$

Para simplificar, se define:

$$\begin{aligned} a &= -d_5A + z - d_1 \\ b &= d_5B - xc_1 - ys_1 \end{aligned} \quad (46)$$

Y queda de la siguiente forma:

$$\begin{aligned} a_3c_{23} &= -a_2c_2 + a \\ a_3s_{23} &= -a_2s_2 - b \end{aligned} \quad (47)$$

Al observar el lado izquierdo de cada ecuación, puede notarse que son las componentes de un vector, por lo tanto, podemos simplificar al elevar al sumar el cuadrado de cada lado entre sí.

$$(a_3c_{23})^2 + (a_3s_{23})^2 = (-a_2c_2 + a)^2 + (-a_2s_2 - b)^2 \quad (48)$$

$$a_3^2 = a_2^2c_2^2 - 2a_2c_2a + a^2 + a_2^2s_2^2 + 2a_2s_2b + b^2 \quad (49)$$

$$a_3^2 = a_2^2 + a^2 + b^2 + 2a_2s_2b - 2a_2c_2a \quad (50)$$

Se define:

$$\begin{aligned} C &= a_2^2 + a^2 + b^2 - a_3^2 \\ D &= 2a_2b \end{aligned} \quad (51)$$

$$E = -2a_2a$$

$$Ds_2 + Ec_2 + C = 0 \quad (52)$$

Se aplica la sustitución por la tangente de medio ángulo, de forma análoga a lo realizado en el primer ángulo y se obtiene:

$$t^2(C - E) + t(2D) + (C + E) = 0 \quad (53)$$

$$t_{1,2} = \frac{-2D \pm \sqrt{4D^2 - 4(C - E)(C + E)}}{2(C - E)} \quad (54)$$

$$\theta_{2-1,2} = 2\arctan(t_{1,2}) \quad (55)$$

Se reutilizan las ecuaciones de 47 para formar:

$$t_{23} = \frac{s_{23}}{c_{23}} = \frac{-a_2s_2 - b}{-a_2c_2 + a} \quad (56)$$

$$\theta_{23} = \operatorname{atan2}(t_{23}) \quad (57)$$

$$\theta_3 = \theta_{23} - \theta_2 \quad (58)$$

Se recuerda que debe mantenerse la trazabilidad de resultados para evitar cruzar soluciones. Finalmente, se reutilizan las ecuaciones de 43:

$$c_{234} = \frac{r_{13}c_1 + r_{23}s_1}{-s_5} \quad (59)$$

$$s_{234} = \frac{-r_{33}}{s_5}$$

$$t_{234} = \frac{r_{33}}{-(r_{13}c_1 + r_{23}s_1)} \quad (60)$$

$$\theta_{234} = \text{atan2}(t_{234}) \quad (61)$$

$$\theta_4 = \theta_{234} - \theta_2 - \theta_3 \quad (62)$$

Se deben tener en cuenta las mismas consideraciones sobre la trazabilidad de soluciones que se ha mencionado previamente.

Se han obtenido todas las soluciones geométricas posibles (8) desde las cuales puede elaborarse un cuadro de referencia para seguir la traza de cada solución más claramente:

	q_1	q_2	q_3	q_4	q_5	q_6
Solución 1	q_1	q_2	q_3	q_4	q_5	q_6
Solución 2	q_1	q_2^I	q_3^I	q_4^I	q_5	q_6
Solución 3	q_1	q_2^{II}	q_3^{II}	q_4^{II}	q_5^I	q_6^I
Solución 4	q_1	q_2^{III}	q_3^{III}	q_4^{III}	q_5^I	q_6^I
Solución 5	q_1^I	q_2^{IV}	q_3^{IV}	q_4^{IV}	q_5^{II}	q_6^{II}
Solución 6	q_1^I	q_2^V	q_3^V	q_4^V	q_5^{II}	q_6^{II}
Solución 7	q_1^I	q_2^{VI}	q_3^{VI}	q_4^{VI}	q_5^{III}	q_6^{III}
Solución 8	q_1^I	q_2^{VII}	q_3^{VII}	q_4^{VII}	q_5^{III}	q_6^{III}

El robot a desarrollar cuenta con articulaciones rotativas con rango $[-2\pi, 2\pi]$, lo que permite realizar un giro completo en cada sentido. Esto permite una mayor flexibilidad en la planificación de trayectorias, ya que existen múltiples configuraciones articulares equivalentes para alcanzar una misma pose del efector final. Esto provoca una *periodicidad en las soluciones*, es decir, se puede alcanzar el mismo punto de un círculo al girar 15° o -345° .

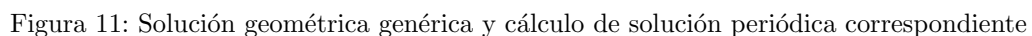
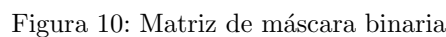
En primer lugar deben conocerse la cantidad de soluciones totales considerando periodicidad en articulaciones, dado que el rango total es, $\pm k\pi$ donde $k = 2$, se duplicarán las soluciones por cada articulación que tenga periodicidad. En este caso, se consideran 6 articulaciones con periodicidad igual a 2 (NOTA: se considera que el rango de la última articulación es $\pm 2\pi$, en realidad es una articulación libre por lo que conduciría a infinitas soluciones), así:

$$\text{Cantidad total de soluciones} = \text{Cantidad de soluciones geométricas} \times 2^6 = 8 \times 2^6 = 512 \quad (63)$$

La metodología elegida para formar soluciones coherentes es:

1. Se define una matriz vacía cuyo tamaño corresponde al número total de soluciones, dado por: *Cantidad de soluciones geométricas* $\times 2^6$.
2. Se construye una matriz máscara binaria de tamaño 64×6 , cuyas filas codifican la selección entre el valor original de cada articulación (0) y su equivalente periódico (1), garantizando así la generación de todas las combinaciones posibles. Esto se itera para cada solución geométrica encontrada.

- A continuación, se presentan imágenes de la forma genérica de cada matriz.



Las singularidades representan configuraciones en las que un manipulador robótico pierde uno o más grados de libertad en el espacio cartesiano, independientemente del movimiento que se imponga en el espacio articular. En un robot de seis grados de libertad, estas configuraciones se identifican a partir del análisis del Jacobiano geométrico $\mathbf{J}(\mathbf{q})$, cuya pérdida de rango completo indica la presencia de una singularidad. Formalmente, una configuración $\mathbf{q} \in \mathbb{R}^6$ es singular si y solo si

$$\det(\mathbf{J}(\mathbf{q})) = 0. \quad (64)$$

En estas configuraciones, el robot exhibe dos consecuencias prácticas críticas: la imposibilidad de generar velocidades cartesianas en determinadas direcciones y la amplificación teóricamente ilimitada de velocidades articulares ante pequeñas velocidades cartesianas en las direcciones próximas al espacio nulo del Jacobiano. Por este motivo, la detección y el tratamiento de singularidades son requisitos indispensables en la planificación de trayectorias y en el control de manipuladores industriales.

Al realizar un análisis geométrico, se identifican tres tipos de singularidades: de hombro, de codo y de muñeca. Matemáticamente, cada una corresponde a una condición específica sobre las variables articulares del robot.

- **Singularidad de muñeca:** ocurre cuando el eje de la articulación 4 y el eje de la articulación 6 se alinean, colapsando un grado de libertad en la orientación del efector final. La condición que la define es

$$q_5 = k\pi, \quad k \in \mathbb{Z}. \quad (65)$$

- **Singularidad de codo:** ocurre cuando los ejes de las articulaciones 2 y 4 se alinean, de modo que ambas articulaciones producen el mismo efecto rotacional y el Jacobiano pierde rango. La condición que la define es

$$q_3 = k\pi, \quad k \in \mathbb{Z}. \quad (66)$$

- **Singularidad de hombro:** ocurre cuando el punto de intersección de los ejes articulares 5 y 6 se encuentra sobre el plano que forman los ejes articulares 1 y 2. La forma analítica de esta explicación es hacer que la proyección vertical total de a_2 , a_3 y d_5 sea nula:

$$a_2 \sin(q_2) + a_3 \sin(q_2 + q_3) + d_5 \sin(q_2 + q_3 + q_4) = 0 \quad (67)$$

La verificación de las singularidades se explica en [A.2](#)

6. Planificación de trayectorias

6.1. Generación de trayectorias

La generación de trayectorias constituye una etapa fundamental en el control de manipuladores robóticos, ya que permite definir la evolución temporal de las variables articulares o del efector final entre configuraciones iniciales y finales previamente establecidas. Este proceso no solo implica determinar el camino geométrico a seguir, sino también garantizar que el movimiento cumpla con restricciones físicas del sistema, tales como límites de velocidad, aceleración y suavidad.

El cálculo de la trayectoria se realiza dentro del script `gen_traj.m` el cual recibe los siguientes argumentos:

- **R**: definición del robot.
- **cpoints_input**: matriz de puntos vía en coordenadas cartesianas, uno por fila, con una séptima columna que indica el modo de interpolación para llegar a ese punto (0: espacio articular, 1: espacio cartesiano).
- **q0**: coordenada articular inicial del robot.
- **qddmax, qddmax**: velocidades y aceleraciones máximas articulares.
- **verbose**: controla la verbosidad de la cinemática inversa interna.
- **vmax, amax, wmax, alphamax**: velocidades y aceleraciones lineales y angulares máximas del efector final.

El trabajo comienza por analizar la séptima columna de cada fila (punto vía) del argumento **cpoints_input**, para determinar en qué modo de interpolación llegar a ese punto: 0 para interpolación en espacio articular y 1 para interpolación en espacio cartesiano.

Si el modo es 0, el primer paso es empaquetar todos los puntos vías consecutivos dentro de la matriz de *waypoints* para obtener una trayectoria suave y continua sin detención en cada punto vía dado originalmente. En segundo lugar, estos puntos se transforman al espacio articular mediante la aplicación del cálculo analítico de la cinemática inversa provisto en el script `inv_kinematics.m`. Los puntos vía en el espacio articular son enviados al interpolador de espacio articular provisto por el toolbox **mstraj**.

En el otro caso, cuando el modo es 1, la interpolación se da en el espacio cartesiano por lo que simplemente se envía el punto objetivo al interpolador en espacio cartesiano provisto en el toolbox *ctray* en conjunto con la ley de movimiento definida en *s*. La ley de movimiento se calcula de modo que el tiempo total del movimiento quede definido por el parámetro más comprometido (*vmax*, *amax*, *wmax*, *alphamax*).

En ambos casos, al calcular la trayectoria, esta se acopla a continuación de la trayectoria ya definida en una matriz. El último elemento de esta matriz se utiliza punto inicial de la siguiente interpolación, en el caso de que esté vacía, es decir, que sea la primera interpolación, se utilizará el argumento **q0**.

La validación del funcionamiento del módulo de generación de trayectorias se presenta en el Apéndice [A.3](#).

6.2. Escalamiento de velocidades y aceleraciones

Una vez generada una trayectoria preliminar, es necesario verificar que el perfil de velocidades y aceleraciones resultante respeta los límites cinemáticos del robot, tanto en el espacio articular como en el espacio cartesiano. Para esto se realiza el siguiente procedimiento de dos etapas.

Primera etapa: generación preliminar y cómputo del factor de escala

Se genera la trayectoria con los parámetros nominales. A continuación, la función auxiliar `compute_scale` calcula numéricamente las velocidades y aceleraciones articulares mediante diferencias finitas y, a través del Jacobiano del robot $\mathbf{J}(\mathbf{q})$, obtiene las velocidades y aceleraciones cartesianas:

$$\mathbf{v} = \mathbf{J}(\mathbf{q}) \dot{\mathbf{q}}, \quad \mathbf{a} = \mathbf{J}(\mathbf{q}) \ddot{\mathbf{q}}. \quad (68)$$

Con estos valores se determina si algún límite es superado y en qué medida. La clave del método reside en la relación entre el factor de escala temporal k y sus efectos sobre la cinemática: si se ejecuta el mismo recorrido en k veces más tiempo, las velocidades se reducen en un factor $1/k$ y las aceleraciones en $1/k^2$. Por esto, los factores de escala individuales se calculan de forma diferente según se trate de una restricción de velocidad o de aceleración:

$$k_{\dot{q}} = \max_i \frac{|\dot{q}_i|_{\max}}{\dot{q}_{i,\max}}, \quad k_{\ddot{q}} = \sqrt{\max_i \frac{|\ddot{q}_i|_{\max}}{\ddot{q}_{i,\max}}}, \quad (69)$$

y de manera análoga para las magnitudes cartesianas:

$$k_v = \frac{\|\mathbf{v}\|_{\max}}{v_{\max}}, \quad k_\omega = \frac{\|\boldsymbol{\omega}\|_{\max}}{\omega_{\max}}, \quad k_a = \sqrt{\frac{\|\mathbf{a}\|_{\max}}{a_{\max}}}, \quad k_\alpha = \sqrt{\frac{\|\boldsymbol{\alpha}\|_{\max}}{\alpha_{\max}}}. \quad (70)$$

La raíz cuadrada en los factores de aceleración refleja que, al estirar el tiempo por k , la aceleración se reduce en k^2 , por lo que para cancelar una violación de factor r en aceleración basta con $k = \sqrt{r}$. El factor de escala final del segmento es:

$$k = \max(1, k_{\dot{q}}, k_{\ddot{q}}, k_v, k_\omega, k_a, k_\alpha). \quad (71)$$

Si $k = 1$ no existe ninguna violación y la trayectoria preliminar es válida.

Segunda etapa: regeneración de la trayectoria

Si $k > 1$, se regenera la trayectoria con el tiempo extendido. En el caso de interpolación articular (**mstraj**), esto equivale a reducir los límites de velocidad articular a $\dot{\mathbf{q}}_{\max}/k$ y extender el tiempo de rampa a $t_{\text{acc}} \cdot k$, con lo que la trayectoria resultante cumple todos los límites. En el caso de interpolación cartesiana, simplemente se extiende el tiempo total a t_f a $k \cdot t_f$ y se regenera la ley de movimiento quintic. En este modo existe además una estimación inicial de t_f basada en las propiedades analíticas del perfil quintic empleado, que sirve como punto de partida para la verificación posterior.

La validación del funcionamiento del escalamiento de velocidades se presenta en el Apéndice [A.3](#).

7. Diseño y adaptación de CAD

El diseño tridimensional del robot constituye una etapa fundamental para la visualización y validación del modelo cinemático desarrollado. Con este objetivo, se elaboraron los modelos CAD de cada eslabón en el entorno *Fusion 360*, exportándolos posteriormente en formato STL para su integración en el entorno de simulación. Dado que las dimensiones y unidades de los archivos exportados no coinciden directamente con los parámetros del modelo DH, se desarrolló el script `scale.m`, que se encarga de escalar cada STL de forma que las dimensiones coincidan con las definidas modelo DH. Adicionalmente, durante el proceso de diseño se buscó maximizar el aprovechamiento del rango articular de cada articulación, procurando que la configuración mecánica permita explotar al máximo las capacidades de movimiento del robot sin incurrir en interferencias geométricas entre sus componentes.

7.1. Visualización del modelo en MATLAB

La figura 12 muestra el ensamblaje completo del robot renderizado en *MATLAB* mediante la toolbox de *Peter Corke*, con los marcos de referencia DH superpuestos sobre cada eslabón y con transparencia aplicada sobre las mallas STL, lo que permite verificar visualmente la alineación entre los ejes articulares y la geometría de cada pieza. Se incluyen también los sistemas del modelo Dh.

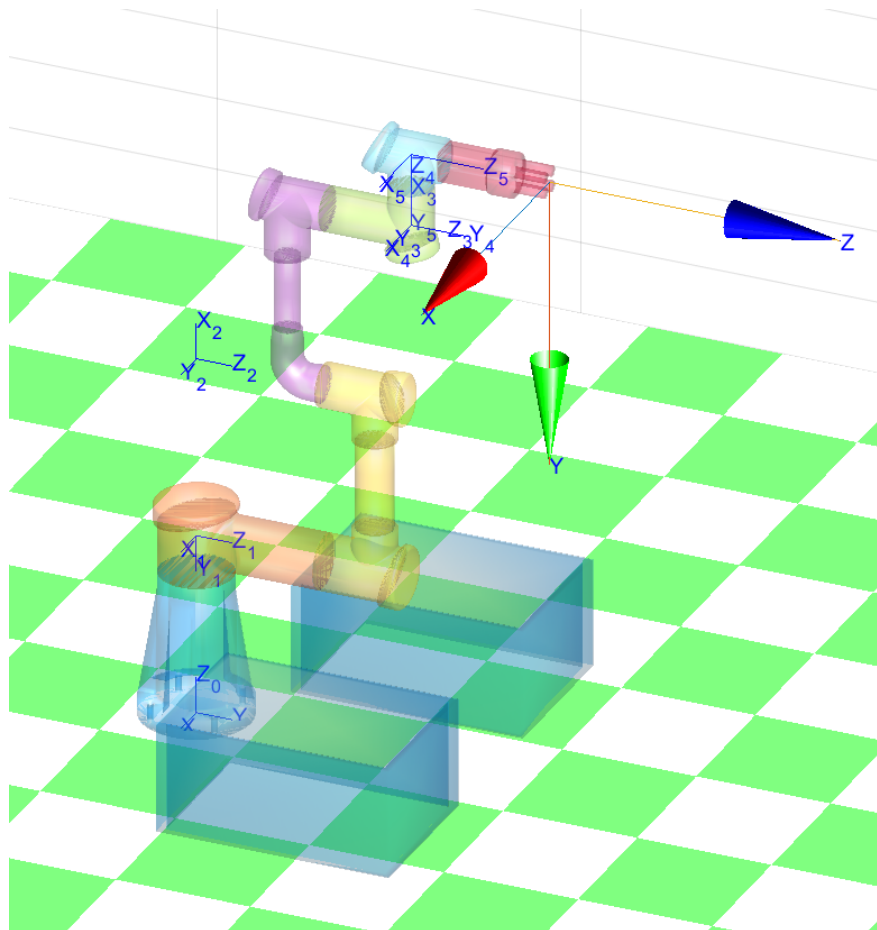


Figura 12: Modelo del robot renderizado en *MATLAB* con marcos de referencia DH.

7.2. Diseño de piezas en *Fusion 360*

A continuación se presentan los modelos de cada pieza diseñada en *Fusion 360*. Para cada componente se detallan las decisiones de diseño adoptadas en relación al aprovechamiento del rango articular y la evitación de interferencias geométricas. En todos los casos, el sistema de referencia de cada cuerpo se adapta de forma que resulte correlativo con el modelo DH, garantizando la coincidencia entre el eje de giro definido en la parametrización cinemática y el cuerpo sobre el cual dicho giro se produce.

7.2.1. Base

Para el diseño de la base se tienen en cuenta dos premisas. En primer lugar, se incrementa el grosor en la zona de apoyo con el objetivo de aumentar la estabilidad del conjunto. En segundo lugar, se incorpora una altura suficiente para posicionar al robot por encima de las cajas, tal como lo requiere la tarea descrita en la sección 3.

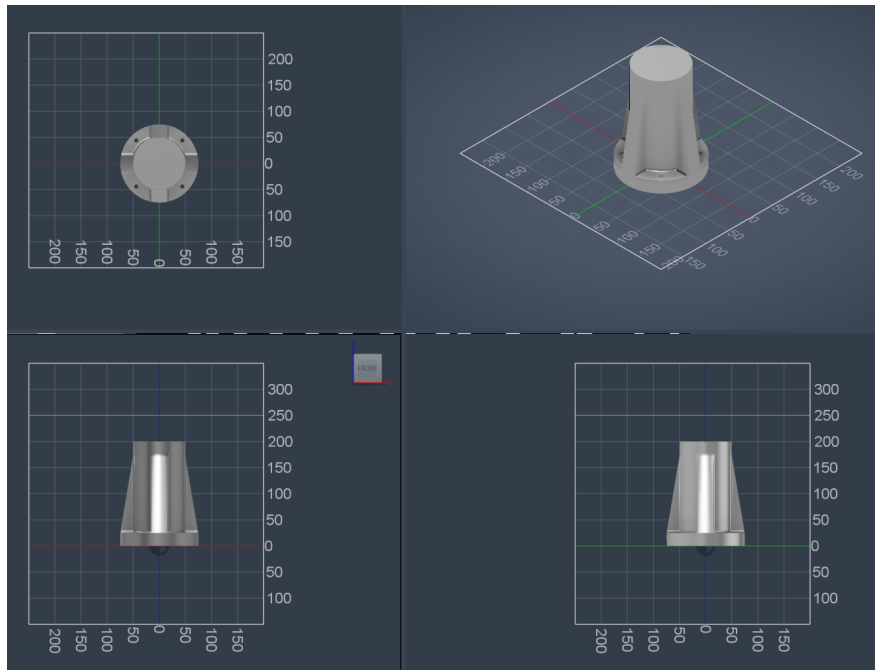


Figura 13: Diseño de la base en *Fusion 360*.

7.2.2. Eslabón 1

El eslabón 1 tiene como objetivo principal proveer una separación del eje central definido por la base, de forma que los eslabones posteriores cuenten con espacio suficiente para aprovechar su rango articular sin incurrir en interferencias geométricas.

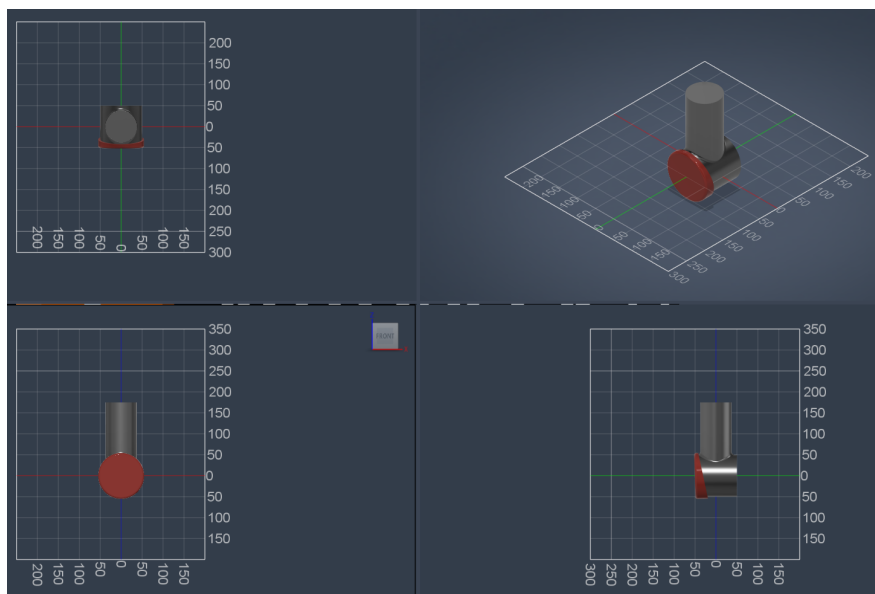


Figura 14: Diseño del eslabón 1 en *Fusion 360*.

7.2.3. Eslabón 2

El eslabón 2 cumple la misma función que el eslabón 1. La separación total requerida se distribuye entre ambos eslabones, logrando un equilibrio en la geometría del robot.

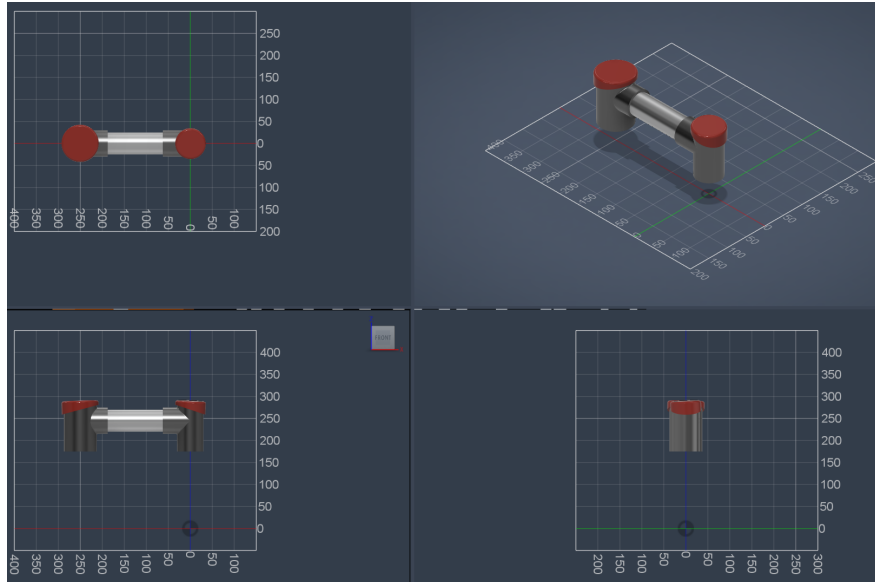


Figura 15: Diseño del eslabón 2 en *Fusion 360*.

7.2.4. Eslabón 3

El eslabón 3 no presenta decisiones de diseño particulares más allá de las consideraciones generales de la sección.

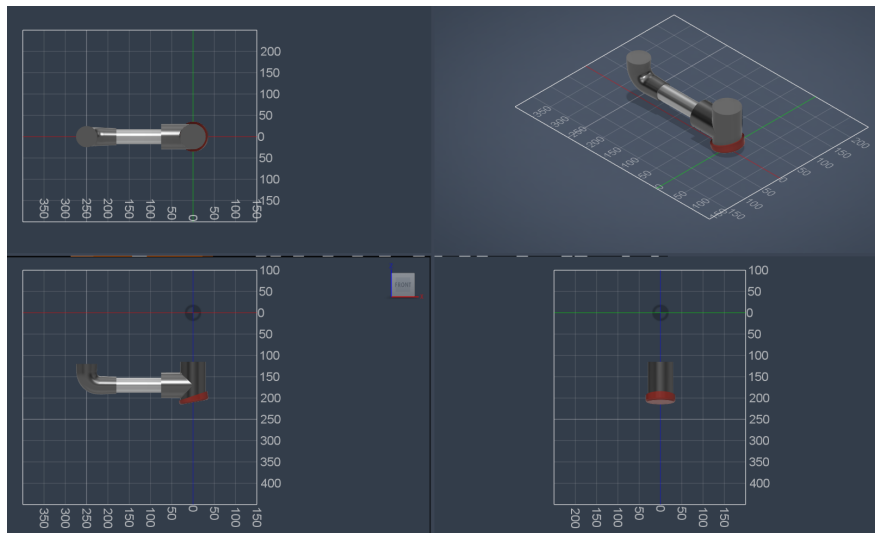


Figura 16: Diseño del eslabón 3 en *Fusion 360*.

7.2.5. Eslabón 4

El eslabón 4 no presenta decisiones de diseño particulares más allá de las consideraciones generales de la sección.

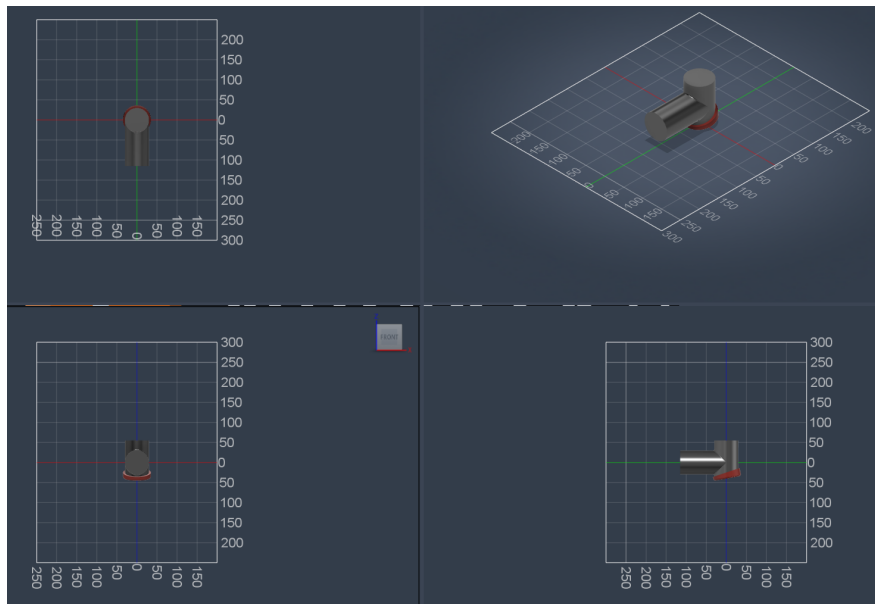


Figura 17: Diseño del eslabón 4 en *Fusion 360*.

7.2.6. Eslabón 5

El eslabón 5 no presenta decisiones de diseño particulares más allá de las consideraciones generales de la sección.

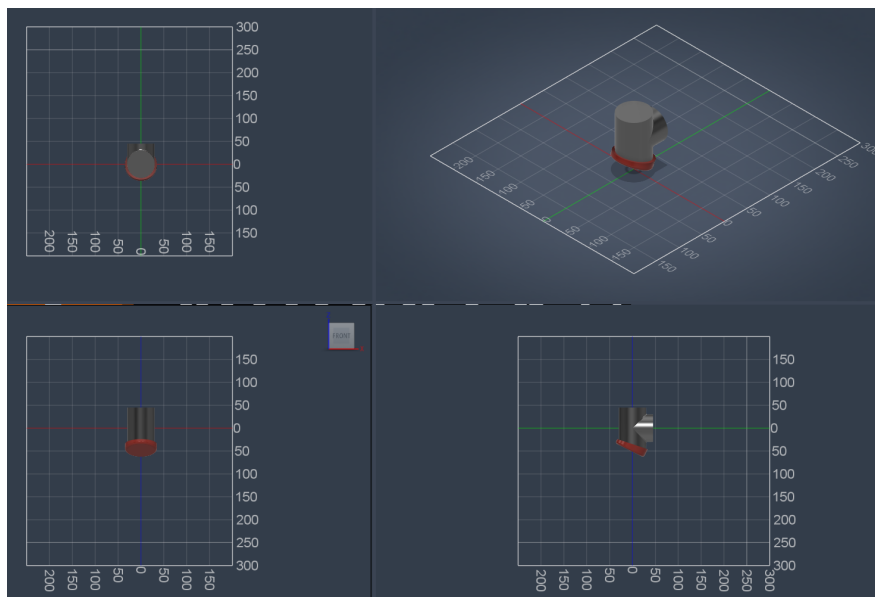


Figura 18: Diseño del eslabón 5 en *Fusion 360*.

7.2.7. Eslabón 6 – Herramienta

El eslabón 6 corresponde a la herramienta del sistema y no presenta decisiones de diseño particulares más allá de las consideraciones generales de la sección.

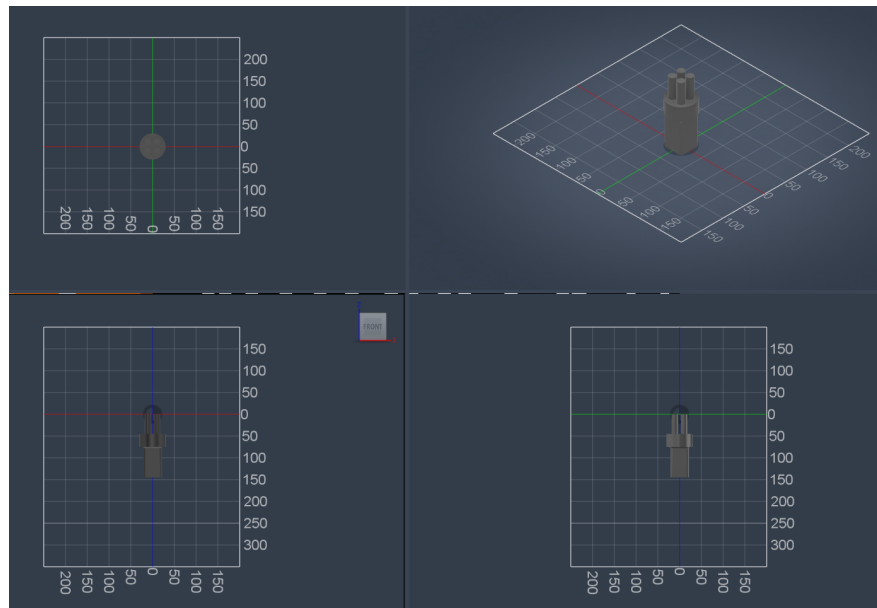


Figura 19: Diseño del eslabón 6 (herramienta) en *Fusion 360*.

7.3. Diseño alternativo: apertura unidireccional de eslabones

En robots de morfología similar al [UR3e](#), los eslabones se disponen de manera tal que, visto desde el plano lateral, el brazo adopta una forma en Z : el eslabón 2 se extiende en un sentido y el eslabón 3 se pliega en sentido opuesto, de forma que los giros de ambas articulaciones se compensan parcialmente entre sí. Esto implica que el aprovechamiento del rango articular de cada junta no es aditivo en la dirección de extensión del brazo.

Como alternativa, se diseñó una segunda versión del robot en la que todos los eslabones se abren en el mismo sentido de giro, adoptando una morfología en C vista desde el plano lateral. De este modo, el aporte de cada articulación al alcance del efector final es aditivo, lo que permite un mejor aprovechamiento del rango articular disponible.

Cabe destacar que esta modificación es estrictamente geométrica: la parametrización cinemática del robot no varía, dado que la definición del modelo Denavit-Hartenberg depende de la disposición de los ejes articulares y no de la forma particular de los eslabones. En consecuencia, toda la cinemática directa e inversa desarrollada en la sección 5 es igualmente válida para ambos diseños.

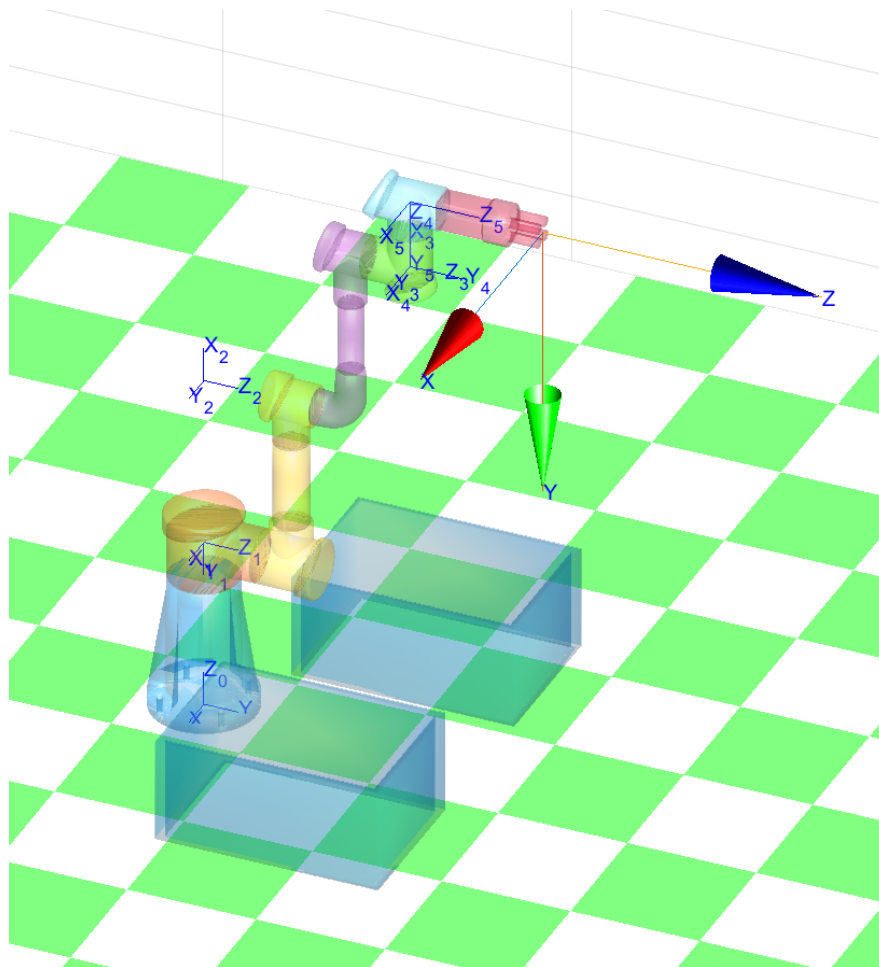


Figura 20: Modelo del robot con diseño alternativo de apertura unidireccional renderizado en *MATLAB*.

8. Simulación y verificación de tarea

Implementando, de forma conjunta el cálculo de la cinemática inversa y el algoritmo de generación de trayectorias, puede desarrollarse una trayectoria para la tarea la cual luego será simulada y analizada.

Para esto se definen los siguientes puntos vía, cuyas orientaciones corresponden a ángulos de Euler ZYX en grados (la herramienta apunta siempre hacia abajo, $\beta = 180^\circ$):

Punto	x [m]	y [m]	z [m]	α [°]	β [°]	γ [°]
caja_origen_superior	-0,22	0,30	0,20	0	180	0
caja_origen_inferior	-0,22	0,30	0,01	0	180	0
aux_1	-0,10	0,30	0,22	0	180	0
punto_medio	0,00	0,30	0,25	0	180	0
aux_2	0,10	0,30	0,22	0	180	0
caja_destino_superior	0,22	0,30	0,20	0	180	0
caja_destino_inferior	0,22	0,30	0,01	0	180	0

Cuadro 1: Puntos vía de la tarea de *bin-to-bin transfer*.

NOTA: En el código se realiza la conversión de grados a radianes y, a los valores angulares, se le aplica una pequeña rotación adicional para evitar definir puntos vía que conduzcan a configuraciones singulares.

El *script* permite el comienzo desde un punto aleatorio o desde un punto conocido. Se recuerda que, en conjunto con los puntos vías debe ser proporcionada la indicación del método de interpolación a utilizar, la secuencia es la siguiente:

1. **Posición inicial a punto medio:** Interpolación articular
2. **Punto medio a caja origen superior:** Interpolación articular
3. **caja origen superior a caja origen inferior:** Interpolación cartesiana
4. **Caja origen inferior a caja origen superior:** Interpolación cartesiana
5. **Caja origen superior a punto medio:** Interpolación articular
6. **Punto medio a caja destino superior:** Interpolación articular
7. **Caja destino superior a caja destino inferior:** Interpolación cartesiana
8. **Caja destino inferior a caja destino superior:** Interpolación cartesiana
9. **Caja destino superior a punto medio:** Interpolación articular

De este modo, se tendrá mayor control en el espacio cartesiano cuando se requiera ingresar y egresar de la caja de recogida y de colocación.

A continuación, se presenta una imagen luego de la simulación:

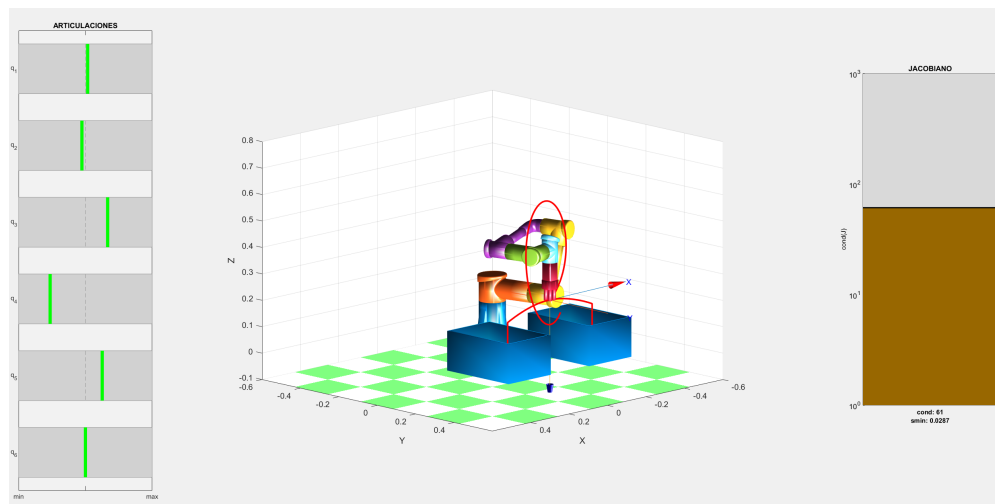


Figura 21: Imagen del final de la simulacion de la tarea.

En la imagen puede verse el recorrido del robot, el cual ha dejado una traza roja de la posición del efector final. En el inicio de la trayectoria puede verse una maniobra que busca posicionar al robot en el llamado punto medio bajo una configuración articular geométrica lo que beneficiará el comportamiento posterior dado que se comenzará lejos de los límites articulares. A los costados del robot, pueden verse gráficas animadas de la posición instantánea de cada articulación y del valor instantáneo del jacobiano lo que permite evaluar diferentes trayectorias.

Luego de la simulación, se obtienen las siguientes gráficas para el análisis de las variables cinemáticas tanto del efector final como de las articulaciones, puede notarse que no se sobrepasan significativamente límites cinemáticos:

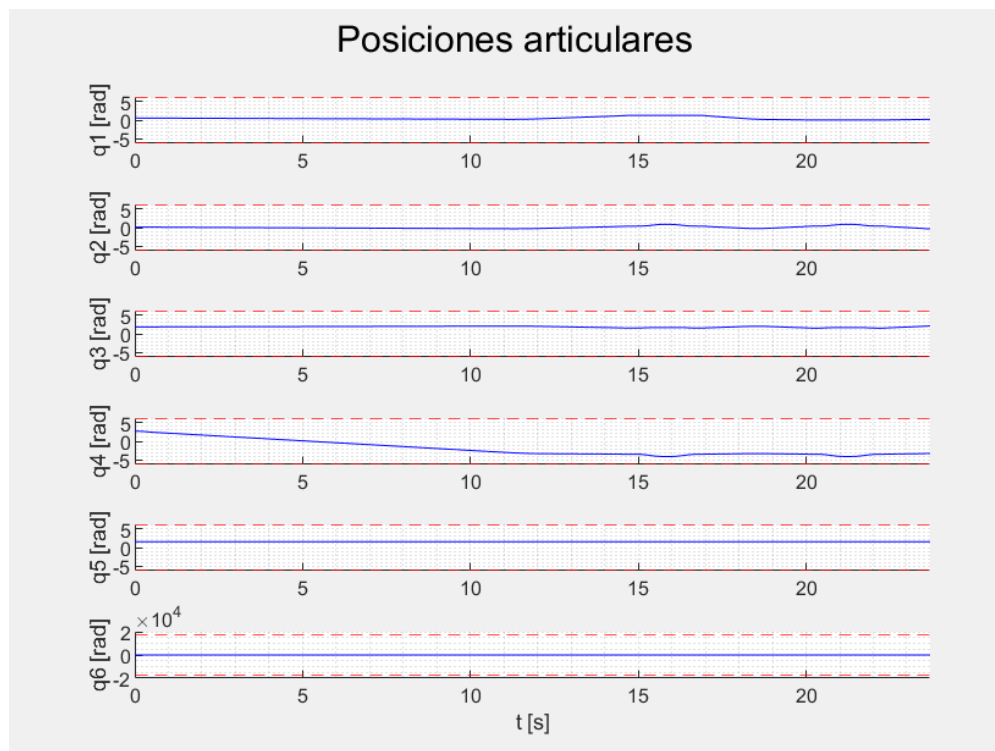


Figura 22: Evolución de posiciones articulares en la simulacion de la tarea.

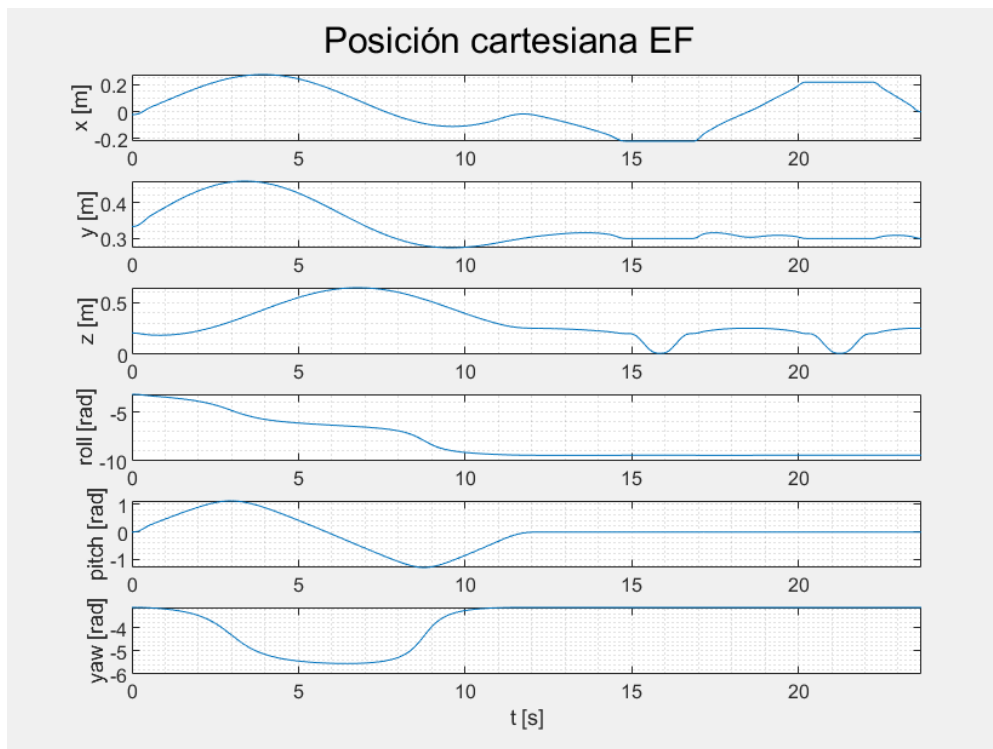


Figura 23: Evolución de la posición cartesiana del efector final en la simulación de la tarea.

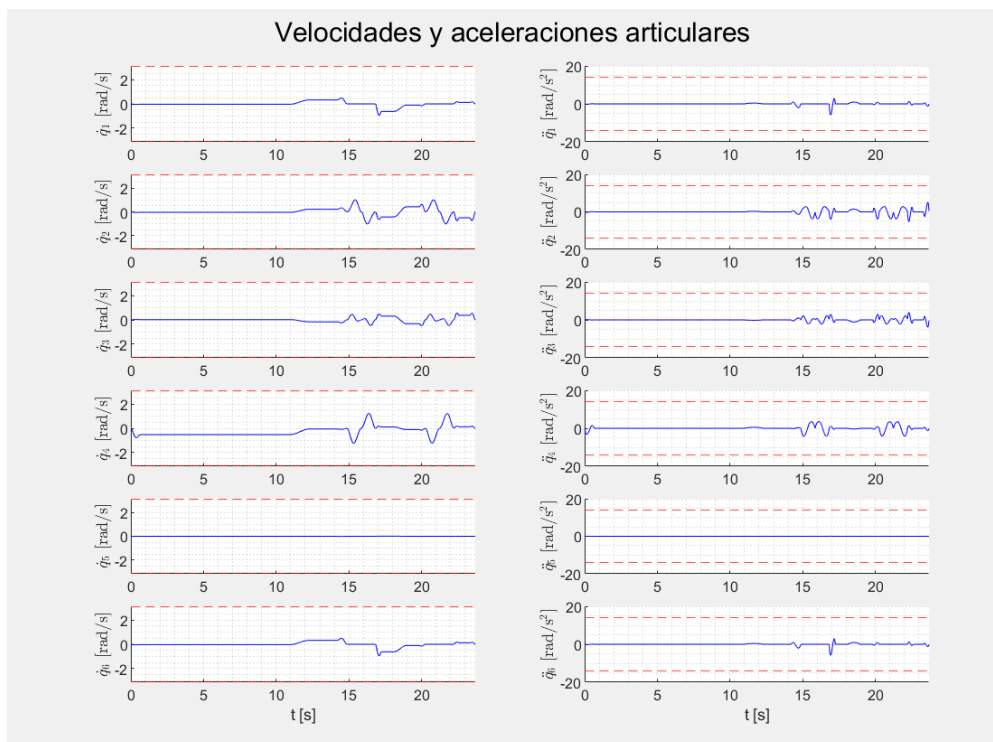


Figura 24: Evolución de velocidades y aceleraciones articulares en la simulación de la tarea.

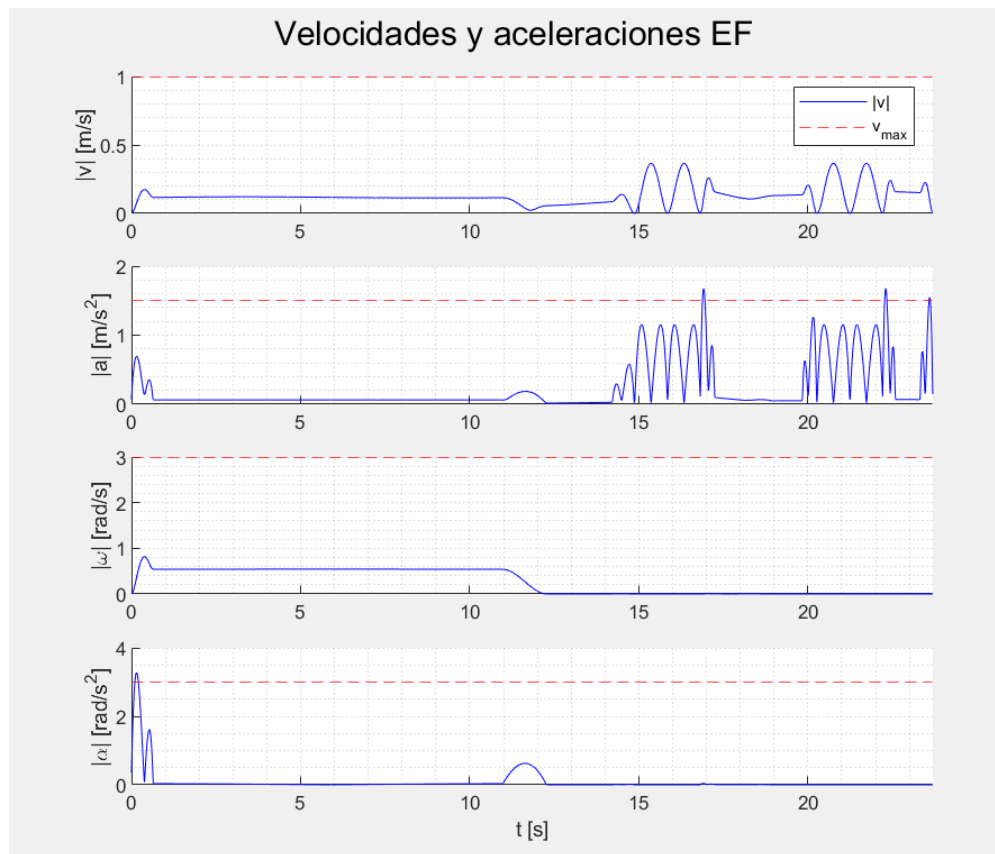


Figura 25: Evolución de velocidades y aceleraciones lineales y angulares del efector final en la simulación de la tarea.

9. Integración en entorno ROS2

En esta sección se describe la integración del manipulador en el entorno *Robot Operating System 2* (ROS 2). El objetivo es trasladar el desarrollo realizado en MATLAB —cinemática directa, cinemática inversa y generación de trayectorias— a una plataforma de software orientada a sistemas robóticos reales, aportando modularidad, comunicación entre procesos y capacidad de simulación mediante RViz. La totalidad del software ha sido desarrollada de forma propia, sin recurrir a librerías de alto nivel específicas de robótica, con el objetivo de maximizar el aprendizaje obtenido y obtener la mayor flexibilidad posible en el diseño de la arquitectura.

La integración se estructura en torno a un conjunto de nodos que encapsulan responsabilidades bien definidas: cálculo cinemático, gestión de trayectorias, *teaching* y visualización a través de una interfaz gráfica. Estos nodos se comunican mediante tópicos y servicios de ROS2, conformando una arquitectura distribuida que permite operar el robot de forma manual o autónoma.

El modelo del robot se describe en formato URDF (*Unified Robot Description Format*), derivado directamente de la parametrización Denavit-Hartenberg definida en la sección 5.1, lo que garantiza consistencia geométrica entre el modelo cinemático y su representación en el entorno de simulación.

9.1. Arquitectura del sistema

En primer lugar, se ha desarrollado un nodo `dr.launcher.py` que se encarga de levantar todos los nodos y enviar, en la mayoría de los casos, los parámetros del robot `robot_params.yml` el cual, no contiene la definición URDF del robot sino parámetros de la cadena cinemáticos, límites cinemáticos y transformaciones de base y de herramienta.

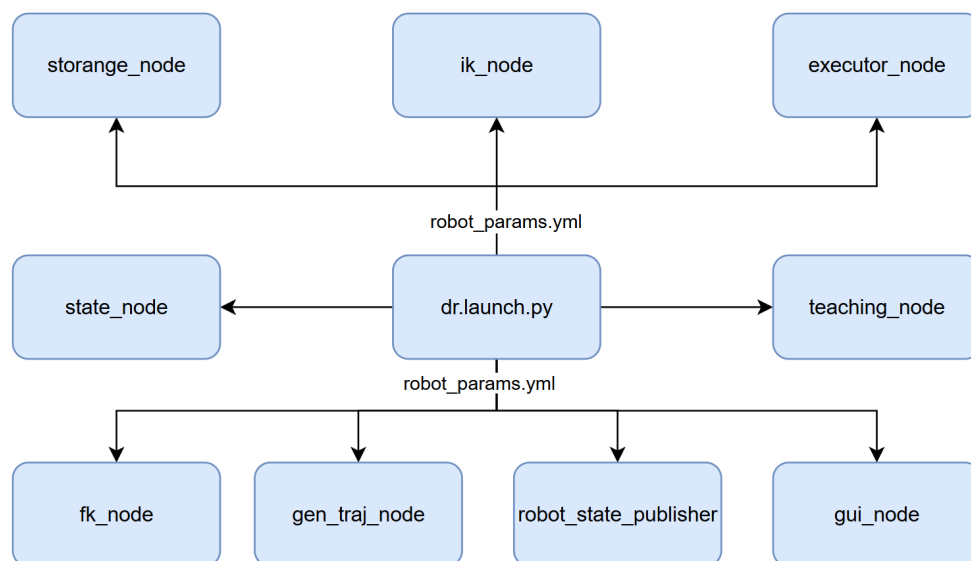


Figura 26: Esquema de levantamiento de nodos desde nodo launcher.

A continuación se describe el propósito e interfaz principal de cada nodo. Los nodos de desarrollo propio se agrupan en paquetes con el prefijo `dr_`; el nodo `robot_state_publisher` es el nodo estándar de ROS 2 que publica las transformaciones del árbol cinemático a partir del URDF y del tópico `/joint_states`.

Cuadro 2: Nodos del sistema ROS 2 y sus interfaces principales.

Nodo	Paquete	Descripción e interfaz principal
<code>fk_node</code>	<code>dr_kinematics</code>	Resuelve la cinemática directa. Expone el servicio <code>/dr/solve_fk</code> : recibe los ángulos articulares $\mathbf{q}$ y devuelve la pose cartesiana del efector final.
<code>ik_node</code>	<code>dr_kinematics</code>	Resuelve la cinemática inversa analítica. Expone el servicio <code>/dr/solve_ik</code> : recibe una pose cartesiana y devuelve el conjunto de soluciones articulares válidas.
<code>gen_traj_node</code>	<code>dr_trajectory</code>	Genera trayectorias en espacio articular o cartesiano con escalado de velocidades. Expone <code>/dr/gen_traj</code> ; consume internamente <code>/dr/solve_ik</code> y <code>/dr/solve_fk</code> . Se suscribe a <code>/dr/robot_state</code> para obtener la configuración inicial.
<code>state_node</code>	<code>dr_state</code>	Agrega el estado articular (<code>/dr/joint_state</code>) y el estado de herramienta (<code>/dr/tool_state</code>), calcula la pose cartesiana o configuración articular actual mediante cinemática directa o inversa según corresponda y publica el estado unificado del robot en <code>/dr/robot_state</code> .
<code>joint_state_bridge_node</code>	<code>dr_state</code>	Convierte <code>/dr/robot_state</code> al mensaje estándar <code>sensor_msgs/JointState</code> publicado en <code>/joint_states</code> , necesario para la visualización en RViz y para el <code>robot_state_publisher</code> .
<code>executor_node</code>	<code>dr_executor</code>	Ejecuta una trayectoria articular recibida como objetivo de la acción <code>/dr/execute</code> , publicando realimentación de posición en <code>/dr/joint_state</code> a lo largo del recorrido.
<code>storage_node</code>	<code>dr_storage</code>	Gestiona la persistencia de trayectorias en formato JSON. Expone los servicios <code>/dr/save_traj</code> , <code>/dr/load_traj</code> , <code>/dr/list_trajs</code> y <code>/dr/delete_traj</code> .
<code>teaching_node</code>	<code>dr_teaching</code>	Convierte incrementos cartesianos recibidos en <code>/dr/teaching_delta</code> en comandos de pose de herramienta publicados en <code>/dr/tool_state</code> , habilitando el movimiento manual incremental del efector final.
<code>gui_node</code>	<code>dr_gui</code>	Interfaz gráfica (PyQt5) que centraliza las funcionalidades del sistema: movimiento manual, enseñanza de puntos, generación y gestión de trayectorias, ejecución y análisis de variables cinemáticas. Se suscribe a <code>/dr/robot_state</code> , publica en <code>/dr/teaching_delta</code> y consume todos los servicios y la acción <code>/dr/execute</code> .

9.2. Interfaces personalizadas

Todas las interfaces de comunicación fueron definidas de forma propia en el paquete `dr_interfaces`. Se clasifican en mensajes, servicios y acciones.

En el caso de las trayectorias articulares, los mensajes de ROS 2 no admiten matrices bidimensio-

nales de tamaño variable, por lo que se adoptó la convención de aplanarlas en vectores unidimensionales (`float64[]`). Cada grupo de seis valores consecutivos corresponde a una configuración articular $\mathbf{q} \in \mathbb{R}^6$, de modo que una trayectoria de N puntos se representa como un vector de $6N$ elementos. El campo entero `points` acompaña siempre a `q_traj` e indica el valor de N , necesario para reconstruir la matriz original al deserializar el mensaje.

Mensajes

Cuadro 3: Mensajes personalizados (`dr_interfaces/msg`).

Mensaje	Campos	Descripción
JointState	float64[6] q float64[6] qd float64[6] qdd	Estado articular completo: posición, velocidad y aceleración de las seis articulaciones.
RobotState	float64[6] q float64[6] qd float64[6] qdd float64[6] pose	Estado unificado del robot: estado articular más pose cartesiana del efector final $(x, y, z, \alpha, \beta, \gamma)$.
ToolState	float64[6] pose	Pose cartesiana de la herramienta $(x, y, z, \alpha, \beta, \gamma)$. Usada para comandar el movimiento del efector en modo <i>teaching</i> .
TeachingDelta	float64 dx, dy, dz float64 droll, dpitch, dyaw	Incremento cartesiano solicitado desde la GUI para el movimiento manual del efector final.

Servicios

Cuadro 4: Servicios personalizados (`dr_interfaces/srv`).

Servicio	Request	Response	Descripción
SolveFK	float64[6] q	float64[6] pose bool success	Cinemática directa.
SolveIK	float64[6] pose float64[6] q0 bool force	float64[6] q bool success	Cinemática inversa analítica.
GenTraj	float64[] pose int8 mode int8 speed_profile	float64[] q_traj int32 points bool success	Generación de trayectoria hacia una pose objetivo.
SaveTraj	string name float64[] q_traj int32 points	bool success	Persistencia de trayectoria en disco.
LoadTraj	string name	float64[] q_traj int32 points bool success	Carga de trayectoria desde disco.
ListTrajs	—	string[] names bool success	Lista de trayectorias almacenadas.
DeleteTraj	string name	bool success	Eliminación de trayectoria del disco.

Acciones

Cuadro 5: Acción personalizada (`dr_interfaces/action`).

Acción	Goal	Feedback	Result
Execute	float64[] q-traj int32 points	float64[6] q, qd, qdd int32 point_index float64 progress	bool success string message

9.3. Interrelación entre nodos

Los nodos del sistema se interrelacionan a través de cuatro flujos funcionales que operan de forma concurrente. A continuación se describe cada uno.

Flujo de estado (permanente). El `state_node` es el nodo central del sistema: agrega toda la información de posición del robot y la redistribuye de forma unificada. Recibe el estado articular desde `executor_node` a través del tópico `/dr/joint_state`, y los comandos de pose de herramienta desde `teaching_node` a través de `/dr/tool_state`. Con esta información, llama al servicio `/dr/solve_fk` del `fk_node` para calcular la pose cartesiana actual y publica el estado completo del robot en `/dr/robot_state`. Este tópico es consumido por `gui_node`, `teaching_node` y `gen_traj_node`. Paralelamente, el `joint_state_bridge_node` convierte `/dr/robot_state` al formato estándar `sensor_msgs/JointState` en `/joint_states`, que es consumido por el `robot_state_publisher` para mantener el árbol de transformaciones TF y habilitar la visualización en RViz.

Flujo de movimiento manual (*teaching*). Cuando el operador acciona los controles de movimiento manual en la GUI, el `gui_node` publica un incremento cartesiano en `/dr/teaching_delta`. El `teaching_node` recibe este incremento y, a partir de la pose actual disponible en `/dr/robot_state`, calcula la nueva pose objetivo y la publica en `/dr/tool_state`. El `state_node` recibe esta actualización, resuelve la cinemática inversa llamando a `/dr/solve_ik` del `ik_node` para obtener la configuración articular correspondiente, y publica el nuevo `/dr/robot_state`, cerrando el ciclo de realimentación hacia la GUI.

Flujo de generación de trayectoria. Cuando el operador solicita generar una trayectoria desde la GUI, el `gui_node` invoca el servicio `/dr/gen_traj` del `gen_traj_node`. Este último consulta `/dr/robot_state` para obtener la configuración articular inicial del robot, y luego llama iterativamente a `/dr/solve_ik` del `ik_node` y a `/dr/solve_fk` del `fk_node` para resolver las configuraciones articulares de cada punto vía y verificar la trayectoria generada. La trayectoria resultante, codificada como vector aplanado, es devuelta al `gui_node` en la respuesta del servicio.

Flujo de ejecución de trayectoria. Una vez disponible la trayectoria, el `gui_node` envía un objetivo (*goal*) a la acción `/dr/execute` del `executor_node`. Este recorre la trayectoria articular punto a punto, publicando en cada paso el estado articular actual en `/dr/joint_state`. El `state_node` procesa estos valores, actualiza `/dr/robot_state` y lo publica, de modo que tanto la GUI como RViz reflejan el movimiento en tiempo real. Simultáneamente, el `executor_node` envía realimentación al cliente de la acción con la posición articular actual, el índice del punto en curso y el progreso normalizado de la ejecución.

Flujo de gestión de trayectorias almacenadas. El `gui_node` interactúa directamente con el `storage_node` mediante los servicios `/dr/save_traj`, `/dr/load_traj`, `/dr/list_trajs` y `/dr/delete_traj` para persistir, recuperar y administrar trayectorias en disco. El `storage_node` no tiene dependencias con ningún otro nodo del sistema, lo que garantiza su operación independiente.

9.4. Modelo URDF

El modelo del robot se describe en formato URDF (*Unified Robot Description Format*) y se aloja en el paquete `robot_description`. Su estructura fue derivada directamente de la parametrización Denavit-Hartenberg definida en la sección 5.1, garantizando consistencia geométrica entre ambas representaciones.

El árbol cinemático está compuesto por un eslabón raíz fijo (`world`), seis eslabones móviles (`link_1` a `link_6`) y un eslabón de herramienta (`tcp`), vinculados mediante seis articulaciones rotativas y una articulación fija. Cada articulación rotativa incorpora los parámetros de offset, traslación y rotación del eslabón correspondiente derivados de la tabla DH, de modo que la cadena cinemática resultante reproduce fielmente el modelo analítico. Las articulaciones 1 a 5 tienen límites de $\pm 2\pi$ rad ($\pm 360^\circ$); la articulación 6 se declara de tipo *continuous* en concordancia con el alcance libre definido en las especificaciones del robot.

El offset de herramienta (T_{tool}) se incorpora como una articulación fija entre `link_6` y `tcp`, con una traslación de 0.190 m en el eje Z, valor consistente con el parámetro `T.tool` de `robot_params.yml`.

Cada eslabón incluye una geometría visual asociada a la malla STL correspondiente del modelo CAD desarrollado en *Fusion 360* (sección 7). Los ejes de referencia de cada malla fueron alineados con los marcos DH durante el proceso de diseño, por lo que no fue necesario aplicar transformaciones adicionales entre el sistema de referencia cinemático y el visual.

Esto puede verse en la siguiente imagen correspondiente al entorno de simulación RViz2:

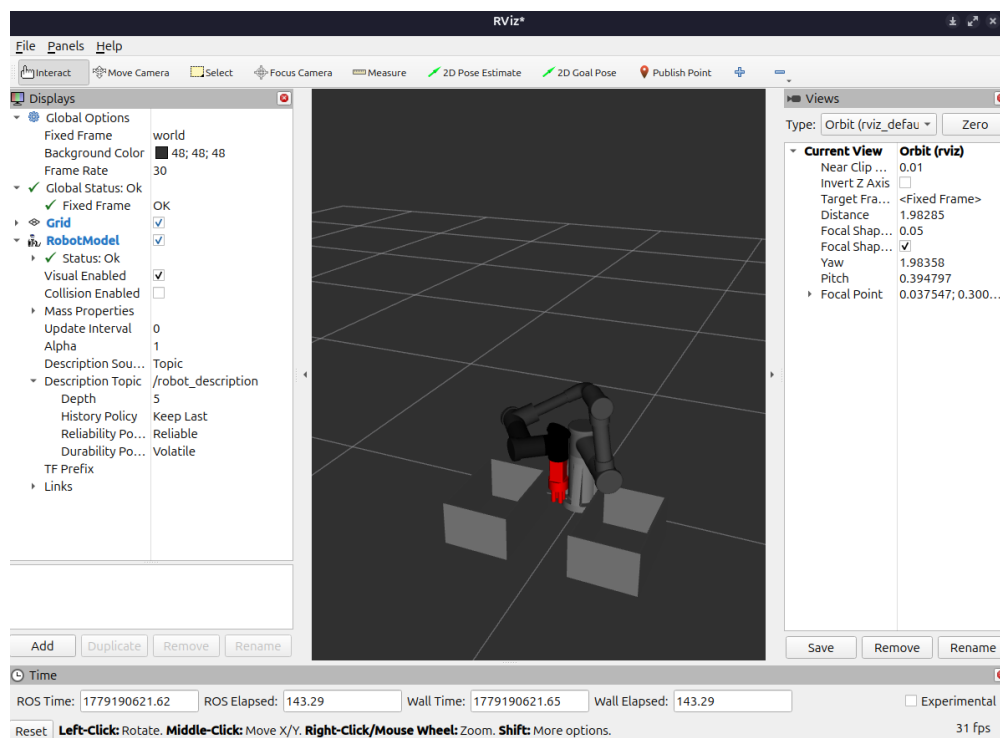


Figura 27: Modelo URDF visualizado en simulador RViz2.

9.5. Cinemática

La cinemática directa e inversa se porta al entorno ROS 2 preservando los algoritmos analíticos desarrollados en la sección 5. Sin embargo, la *Robotics Toolbox* de Peter Corke, utilizada en MATLAB para operaciones de álgebra matricial homogénea y conversión de representaciones de orientación, no está disponible en el entorno Python de ROS 2.

Para resolver esta dependencia se implementó el módulo `transforms.py`, alojado en el paquete `dr_kinematics`. Este archivo replica las funciones del toolbox empleadas en el desarrollo, sin dependencias externas más allá de `numpy`:

- `transl(x, y, z)`: matriz de transformación homogénea de traslación pura.

- $\text{trotx}(\theta)$, $\text{troty}(\theta)$, $\text{trotz}(\theta)$: rotaciones elementales alrededor de los ejes X , Y y Z .
- $\text{inv.T}(\mathbf{T})$: inversa de una transformación homogénea, calculada eficientemente mediante R^T y $-R^T \mathbf{p}$.
- $\text{rot_to_rpy}(\mathbf{R})$: extrae los ángulos de Euler ZYX (roll, pitch, yaw) desde una matriz de rotación, equivalente a tr2rpy del toolbox.

Los parámetros cinemáticos del robot (tabla DH, límites articulares, transformaciones de base y herramienta) se cargan en tiempo de ejecución desde `robot_params.yml`, sin estar codificados en los nodos. Esto permite ajustar el modelo cinemático sin recompilar el código.

9.6. Generación de trayectorias

El módulo de generación de trayectorias se porta al nodo `gen_traj_node`, preservando los dos modos de interpolación: espacio articular (modo 0) y espacio cartesiano (modo 1), y el mecanismo de escalado de velocidades y aceleraciones descrito en la sección 6.2.

El servicio `/dr/gen_traj` acepta **una única pose cartesiana objetivo** por solicitud, junto con el modo de interpolación y un perfil de velocidad (lento, medio o rápido). Al recibir la solicitud, el nodo obtiene la configuración articular actual desde `/dr/robot_state` y genera la trayectoria desde esa configuración hasta el punto objetivo.

Esta decisión de diseño introduce una **limitación respecto a la implementación en MATLAB**: en `gen_traj.m`, la función recibía la lista completa de puntos vía y generaba una única trayectoria continua a través de todos ellos, aprovechando la capacidad de `mstraj` de interpolar sin detención en los puntos intermedios. En ROS 2, dado que el servicio acepta un solo punto por solicitud, cada llamada genera una trayectoria independiente con velocidad inicial y final nula. Como consecuencia, **el robot se detiene completamente en cada punto vía** antes de recibir la siguiente solicitud, lo que impide la interpolación fluida entre segmentos consecutivos.

Esta limitación podría resolverse en el futuro rediseñando el servicio para aceptar una lista de puntos vía en una única solicitud, replicando el comportamiento de `mstraj` en el entorno ROS 2.

9.7. Teaching

El modo *teaching* permite desplazar el efector final de forma incremental desde la interfaz gráfica, capturando configuraciones intermedias que luego se encadenan para construir una trayectoria. El mecanismo se divide en dos componentes: el nodo de *teaching* y la pestaña dedicada en la interfaz gráfica.

Nodo de teaching

El `teaching_node` actúa como un convertidor de desplazamientos relativos a poses absolutas. Al iniciarse, se suscribe a `/dr/robot_state` para obtener y actualizar continuamente la configuración articular actual $\mathbf{q}_0$ y la pose cartesiana correspondiente $\mathbf{c}_0 = [x, y, z, r, p, y]^T$.

Cuando la interfaz gráfica publica un incremento en `/dr/teaching_delta` — mensaje `TeachingDelta` con campos `dx`, `dy`, `dz`, `droll`, `dpitch`, `dyaw` — el nodo suma dicho incremento a $\mathbf{c}_0$ y publica la pose absoluta resultante en `/dr/tool_state`:

$$\mathbf{c}_{\text{objetivo}} = \mathbf{c}_0 + \Delta \mathbf{c}$$

Este enfoque garantiza que cada desplazamiento sea relativo a la pose *actual* del robot, independientemente de si el robot se ha movido entre pulsos de botón.

Pestaña de teaching en la GUI

La pestaña *Teaching* de la interfaz gráfica centraliza el flujo de trabajo de enseñanza de puntos. Su estructura es la siguiente:

- **Tabla de estado actual.** Muestra en tiempo real la configuración articular $\mathbf{q}$ (en radianes) y la pose cartesiana $[x, y, z, \text{roll}, \text{pitch}, \text{yaw}]$ del efector final. Los valores se actualizan cada vez que el nodo recibe un mensaje en `/dr/robot_state`.

- **Controles de jog.** Para cada uno de los seis grados de libertad cartesianos (X, Y, Z, Roll, Pitch, Yaw) se dispone de un par de botones + y --. Al presionar cualquiera de ellos, la función `send_delta(axis, sign)` construye un mensaje `TeachingDelta` con el incremento correspondiente al eje seleccionado y lo publica en `/dr/teaching_delta`. El nodo de *teaching* procesa el delta y publica la nueva pose objetivo en `/dr/tool_state`, que a su vez el nodo de cinemática inversa resuelve y envía al estado del robot.
- **Paso de jog (*step*).** Un *spinbox* configurable entre 0,001 y 0,1 m/rad determina el módulo del incremento aplicado en cada pulsación. Este valor se utiliza directamente en `send_delta`.
- **Modo de interpolación y velocidad.** Un combo de modo (*Articular / Cartesiano*) y un combo de velocidad (*Lento / Medio / Rápido*) acompañan cada waypoint capturado, de modo que la trayectoria resultante puede combinar segmentos con diferentes parámetros.
- **Captura de posición.** El botón “Capturar posición” lee la pose actual de la tabla de estado y la almacena en la lista interna de waypoints junto con el modo y la velocidad seleccionados en ese instante.
- **Gestión de waypoints.** El botón “Ver lista de waypoints” abre un diálogo (`WaypointDialog`) que muestra todos los puntos capturados. Desde este diálogo el usuario puede eliminar puntos individuales antes de confirmar la trayectoria.
- **Finalización.** El campo de texto de nombre y el botón “Finalizar” desencadenan el proceso de generación completo: se itera sobre la lista de waypoints llamando al servicio `/dr/gen_traj` para cada par de puntos consecutivos, las trayectorias parciales se concatenan y el resultado se persiste en disco llamando al servicio `/dr/save_traj`.

9.8. Interfaz gráfica

La interfaz gráfica fue desarrollada con *PyQt5* e integra en un único proceso tanto la lógica de comunicación con ROS 2 como la presentación visual. Para mantener la responsividad de la interfaz, el *spin* de ROS 2 se ejecuta en un hilo demonio (`threading.Thread`), mientras que todas las actualizaciones de widgets se realizan mediante señales Qt (`pyqtSignal`), evitando accesos al hilo principal desde el hilo de ROS 2. Las llamadas a servicios, que son bloqueantes, se realizan también en hilos separados sincronizados mediante `threading.Event`.

La ventana principal (`MainWindow`) se organiza en dos pestañas: *Teaching* (descrita en la sección 9.7) y *Ejecutar*.

Pestaña de ejecución

La pestaña *Ejecutar* permite cargar, recorrer y analizar trayectorias almacenadas. Sus componentes principales son:

- **Tabla de estado actual.** Idéntica a la de la pestaña *Teaching*: muestra **q** y la pose cartesiana en tiempo real.
- **Tabla de trayectorias disponibles.** Cada fila corresponde a una trayectoria almacenada e incluye columnas de nombre, fecha de creación, número de puntos y dos acciones:
 - *Recorrer*: carga la trayectoria llamando a `/dr/load_traj` y envía una solicitud a la acción `/dr/execute`. Durante la ejecución, la pestaña *Teaching* se deshabilita para evitar conflictos.
 - *Eliminar*: solicita confirmación al usuario y, de confirmarse, llama a `/dr/delete_traj` y refresca la tabla.
 - *Analizar*: habilitado únicamente tras ejecutar la trayectoria correspondiente. Abre el diálogo de análisis descrito a continuación.
- **Barra de progreso y etiqueta de estado.** Se actualizan en tiempo real a partir de los mensajes de *feedback* de la acción `/dr/execute`, que reportan el índice del punto en curso. Al completarse la acción, la etiqueta refleja el resultado (*éxito* o *error*).

A continuación, se presentan imágenes correspondientes a cada pestaña.

DR - Panel de Control

Teaching Execute

	q1 / X	q2 / Y	q3 / Z	q4 / Roll	q5 / Pitch	q6 / Yaw
q (rad)	0.5915	0.1403	1.8593	2.7266	1.5736	-0.9894
pose	-0.0316	0.3335	0.2050	-3.1316	-0.0101	-3.1315

X: [-] [+] Step (m / rad):

Y: [-] [+] 0.010

Z: [-] [+] Modo:

Roll: [-] [+] Joint

Pitch: [-] [+] Speed profile:

Yaw: [-] [+] Lento

Ver lista de waypoints

Capturar posición

Nombre de trayectoria: ej: trayectoria_1

Finalizar

Figura 28: Interfáz gráfica de usuario en ROS 2 - Pestaña de teaching.

DR - Panel de Control

Teaching Execute

	q1 / X	q2 / Y	q3 / Z	q4 / Roll	q5 / Pitch	q6 / Yaw
q (rad)	0.5915	0.1403	1.8593	2.7266	1.5736	-0.9894
pose	-0.0316	0.3335	0.2050	-3.1316	-0.0101	-3.1315

	Nombre	Fecha	Puntos	Acciones	Analizar
1	cuadrado_v2	2026-05-14 18:26	804	Recorrer Eliminar	Analizar
2	cuadrado_v1	2026-05-14 18:19	419	Recorrer Eliminar	Analizar

Actualizar lista

Esperando comando...

0%

Figura 29: Interfáz gráfica de usuario en ROS 2 - Pestaña de ejecución.

Diálogo de análisis

Al finalizar la ejecución de una trayectoria, el botón *Analizar* abre el *AnalysisDialog*, un diálogo con cuatro pestañas de visualización:

1. **Coordenadas articulares:** evolución temporal de $q_1, \dots, q_6$ junto con los límites articulares $\pm q_{\max}$ y el perfil de velocidades y aceleraciones articulares con sus respectivos límites.
2. **Posición cartesiana del EF:** evolución de x, y, z y los ángulos de Euler (roll, pitch, yaw) computados mediante cinemática directa.
3. **Velocidad y aceleración lineal:** norma de la velocidad y aceleración lineales del efector final, calculadas a través del Jacobiano geométrico, con comparativa frente a los límites $v_{\max}$ y $a_{\max}$.
4. **Velocidad y aceleración angular:** norma de la velocidad y aceleración angulares del efector final, con comparativa frente a $\omega_{\max}$ y $\alpha_{\max}$.

El cálculo de cinemática directa para todas las muestras de la trayectoria se realiza en un hilo de fondo antes de renderizar los gráficos, de modo que la interfaz no se bloquea durante el proceso. Los gráficos se incrustan directamente en las pestañas del diálogo mediante *FigureCanvas* de *Matplotlib*, sin abrir ventanas externas.

Las siguientes imágenes muestran las 4 pestañas de análisis:

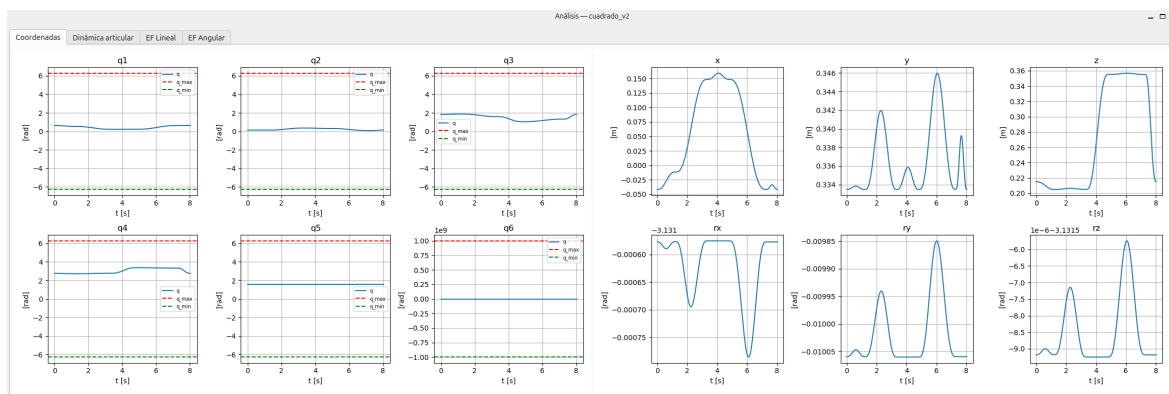


Figura 30: Interfaz gráfica de usuario - Pestaña de análisis: evolución de coordenadas cartesianas del efector final y coordenadas articulares de articulaciones.

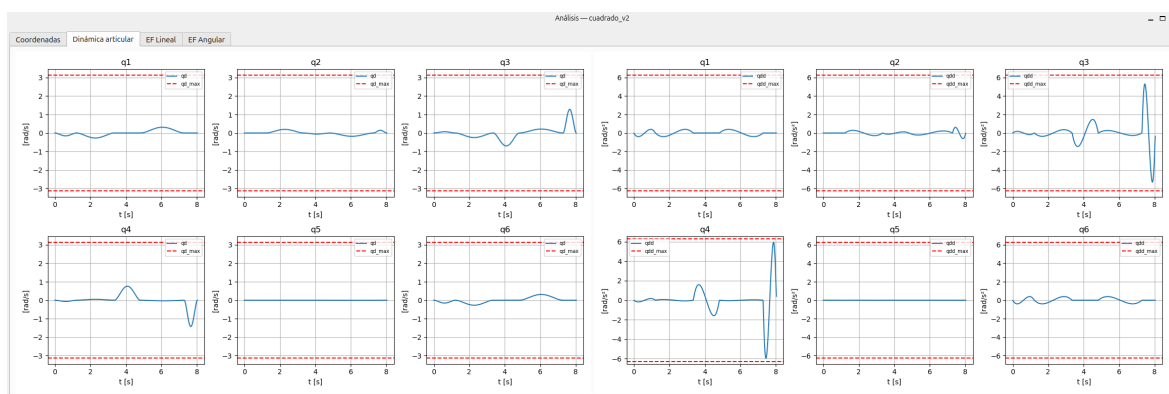


Figura 31: Interfaz gráfica de usuario - Pestaña de análisis: evolución de de velocidad articular y aceleración articular.

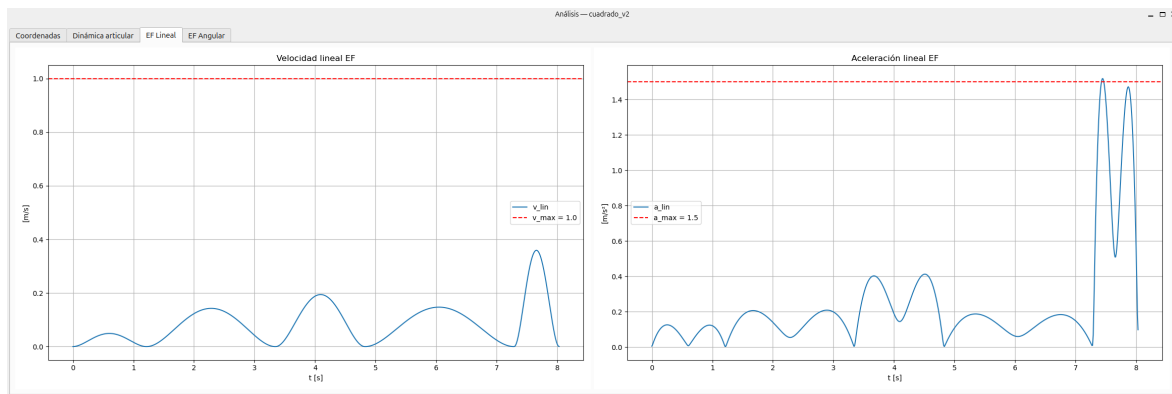


Figura 32: Interfaz gráfica de usuario - Pestaña de análisis: evolución de velocidad lineal y angular del efector final.

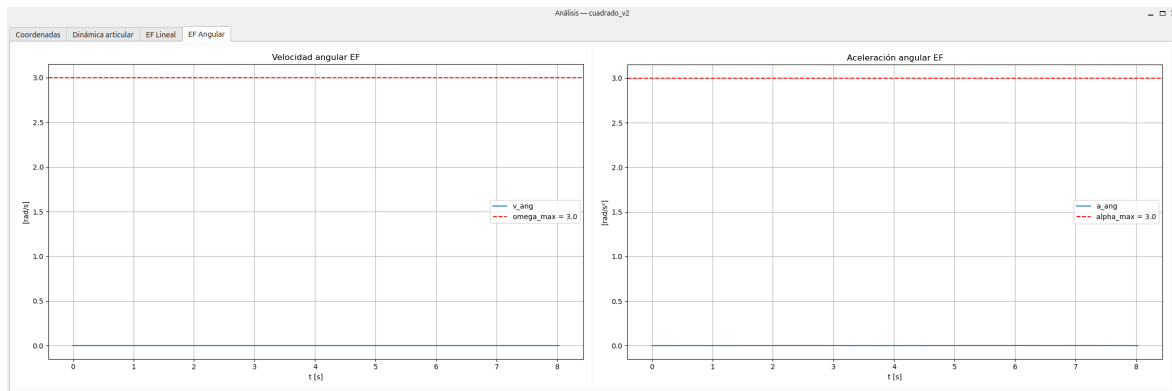


Figura 33: Interfaz gráfica de usuario - Pestaña de análisis: evolución de aceleración lineal y angular del efector final.

De la misma manera al análisis en MATLAB, se presentan los valores obtenidos en conjunto con sus máximos definidos para poder verificar el funcionamiento.

10. Conclusiones y trabajo futuro

A. Verificación y Validación de Módulos de Software de MATLAB y ROS 2

A.1. Cinemática Inversa

La verificación del correcto funcionamiento del módulo de cinemática inversa se realiza mediante el script `test_inv_kinematics.m`. Este incluye 3 funcionalidades

1. **Verificación redundante:** Se define una coordenada articular, se calcula la cinemática directa para obtener la coordenada cartesiana del efector final, sobre esta última se calcula la cinemática inversa con el módulo desarrollado. Finalmente, se presentan los resultados dejando en evidencia la coordenada articular inicial y el conjunto obtenido mediante cinemática inversa, también se calcula la cinemática directa para cada solución obtenida permitiendo otro contraste de resultados.

```
% ==INPUT ==
% Coordenadas articulares ingresadas:
q = [1.8358    0.4922   -1.9151   -0.5557    1.7815    1.2078];
% Las coordenadas cartesianas consigna (calculadas mediante DK) son:
x = -0.16918 | y = -0.19848 | z = 0.29694 | alpha = 2.7989 | beta =
    -0.3074 | gamma = 2.2087
% ==OUTPUT==
% Las coordenadas articulares solución obtenidas de la cinemática
  inversa son (solo algunas):
1.8358    0.4922   -1.9151   -0.5557    1.7815    1.2078
-4.4474    0.4922   -1.9151   -0.5557    1.7815    1.2078
-4.4474   -0.2860   -1.5354    2.9844    4.5017    4.3494
3.2637    0.2942   -4.8523   -3.1284    1.9973    2.7157
% Las coordenadas cartesianas solución obtenidas son las siguientes (se
  presentan solo algunas, todas coinciden):
x = -0.16918 | y = -0.19848 | z = 0.29694 | alpha = 2.7989 | beta =
    -0.3074 | gamma = 2.2087
x = -0.16918 | y = -0.19848 | z = 0.29694 | alpha = 2.7989 | beta =
    -0.3074 | gamma = 2.2087
x = -0.16918 | y = -0.19848 | z = 0.29694 | alpha = 2.7989 | beta =
    -0.3074 | gamma = 2.2087
x = -0.16918 | y = -0.19848 | z = 0.29694 | alpha = 2.7989 | beta =
    -0.3074 | gamma = 2.2087
```

2. **Cálculo directo:** Esta funcionalidad no se define específicamente para realizar una verificación sino para conocer las coordenadas articulares solución de la cinemática inversa a partir de la coordenada cartesiana ingresada de forma directa. Los resultados se presentan de misma forma al ítem 1 dado que solo se realiza un *override* durante ejecución de la coordenada cartesiana de entrada.
3. **Verificación cíclica:** Esta funcionalidad permite, hasta que se detenga la ejecución del programa, verificar cíclicamente puntos aleatorios mediante un esquema de verificación similar al del ítem 1. De forma adicional, se calcula la norma entre coordenadas cartesianas de salida y de llegada, ante una diferencia mayor a un umbral determinado en cualquiera de las soluciones pertenecientes al conjunto de solución obtenido, se detiene el programa para verificar el error y poder analizarlo.

```
% ==INPUT ==
% LOOP -> Probando posición articular
-3.2460   -1.2075   -5.0711   -4.6248    5.5550   -6.2832

% LOOP -> Las coordenadas cartesianas consigna son:
x = 0.10246 | y = -0.40972 | z = 0.45372 | alpha = 3.0035 | beta =
    0.83783 | gamma = 1.3635
% ==OUTPUT==
% Las coordenadas articulares solución obtenidas de la cinemática
  inversa son (solo algunas):
```

```
3.0372    0.7669   -1.7414   -0.5041    0.7282   -3.1416
-3.2460   -5.5163    4.5418    5.7791   -5.5550   -3.1416
-3.2460   -6.2786    5.0711   -3.4127   -0.7282    0
-2.2848    0.0199    0.9305   -3.2374    1.0729   -1.9493
```

```
% Resultado de norma:
```

```
OK
```

```
OK
```

```
OK
```

```
OK
```

NOTA: En los resultados presentados, se mencionan solo una pequeña porción de toda la información devuelta por el script.

A.2. Singularidades

Para verificar que los patrones que conducen a singularidades determinados visualmente son correctos, se elabora el script `test.singularities.m`. En él se encuentra un ciclo *while* infinito, solo terminable mediante finalización de ejecución.

Para la singularidad de codo y de muñeca, se definen valores aleatorios para todas las variables articulares que no estén involucradas en el patrón de la singularidad, para las que sí están involucradas, se define un valor aleatorio dentro del conjunto de aquellos valores que puedan provocar la singularidad según las condiciones definidas en 5.4.

Para la singularidad de hombro, debido a la presencia de 3 incógnitas en una misma ecuación, simplemente se busca verificar la observación, para esto, se dan valores aleatorios dentro de un rango de ± 10 y con signo opuesto para q_2 y q_3 , finalmente se busca el valor de q_4 que cumpla con la condición de proyección nula. El motivo por el que los valores para q_2 y q_3 están dentro del rango ± 10 y de signo opuesto, es para evitar que la proyección no sea anulable geoméricamente mediante q_4 .

Para cada caso, se calcula el jacobiano en la configuración determinada y su determinante, solo en caso de que el determinante del jacobiano sea mayor al valor de umbral, de modo que no se considere cercano a una singularidad, se detiene la ejecución y se muestra información, caso contrario, la ejecución se repite cíclicamente informando cuando se completa cada ciclo. Un ciclo equivale al análisis de una configuración para el codo, una para la muñeca y una para el hombro.

A continuación se detalla un ejemplo para cada salida.

```
% == OUTPUT DE SINGULARIDAD VERIFICADA (TODAS CONFIGURACIONES SINGULARES)
==
Ciclo completo - todas las posiciones fueron singulares - continuando...
% == OUTPUT DE NO SINGULARIDAD ==
Posición articular aleatoria - no singular - hombro
1.7748   -0.1075    0.0635    0.4317   -0.3929    3.0684
Determinante del Jacobiano: 7.897e-20
-> Detención por posición no singular - enter para continuar
```

El último ejemplo, correspondiente a una salida negativa, se produjo exigiendo un determinante de jacobiano excesivamente pequeño para que la verificación falle intencionalmente.

A.3. Generación de trayectorias

En la generación de trayectorias se buscan verificar dos aspectos fundamentales: que la trayectoria generada efectivamente interpole entre los puntos vía definidos y que el perfil de velocidades y aceleraciones resultante respete los límites cinemáticos establecidos. Para esto se elabora el script `test_gen_traj.m` que contiene el siguiente procedimiento de verificación: Se definen manualmente una serie de puntos vía con sus respectivos modos de interpolación (articular o cartesiano) y se genera la trayectoria utilizando el módulo desarrollado y se verifica:

1. **Interpolación entre puntos vía:** Se verifica que la trayectoria resultante efectivamente pase por cada uno de los puntos vía definidos, para esto, se calcula la distancia entre cada punto vía y el punto de la trayectoria correspondiente al mismo tiempo, verificando que esta distancia sea menor a un umbral determinado. En caso de detectar una cercanía lo suficientemente pequeña,

se considera que el punto vía ha sido interpolado correctamente y se incrementa un contador. La verificación será positiva siempre que el contador sea mayor o igual al número total de puntos vía definidos.

2. **Cumplimiento de límites cinemáticos:** Se imprime la comparativa entre valores máximos, perfiles cinemáticos iniciales y perfiles cinemáticos escalados para cumplir con límites. Se verifica: velocidad y aceleración angular de articulaciones, velocidad y aceleración lineal y angular de efector final. De este modo, la verificación será visual.

A.4. Integración en ROS 2

La verificación del sistema en ROS 2 se realizó de forma progresiva a lo largo del desarrollo, probando cada nodo de forma aislada antes de integrarlo con el resto del sistema. La metodología consistió en llamar directamente a los servicios y tópicos desde la terminal, sin depender de la interfaz gráfica, lo que permitió validar cada componente de forma independiente y detectar errores de comunicación, serialización y lógica en etapas tempranas.

A continuación se detallan los comandos más representativos utilizados durante este proceso:

- **Inspección del grafo de nodos y tópicos activos:**

```
ros2 node list
ros2 topic list
ros2 service list
```

- **Monitoreo del estado del robot en tiempo real** (pose articular y cartesiana publicada por el nodo de cinemática):

```
ros2 topic echo /dr/robot_state
```

- **Llamada al servicio de cinemática directa** con una configuración articular manual:

```
ros2 service call /dr/solve_fk dr_interfaces/srv/SolveFK {q
: [0.0, 0.5, -1.0, 0.0, 1.5, 0.0]}
```

- **Llamada al servicio de cinemática inversa** a partir de una pose cartesiana objetivo:

```
ros2 service call /dr/solve_ik dr_interfaces/srv/SolveIK {pose
: [-0.22, 0.3, 0.2, 0.01, 3.1416, 0.01], q0: [0.0, 0.0, 0.0, 0.0, 1.5708, 0.0], force: false
}
```

- **Generación de una trayectoria** desde la configuración actual hasta un punto cartesiano objetivo:

```
ros2 service call /dr/gen_traj dr_interfaces/srv/GenTraj {pose
: [0.0, 0.3, 0.25, 0.01, 3.1416, 0.01], mode: 1, speed_profile: 1}
```

- **Guardado y listado de trayectorias** en el nodo de almacenamiento:

```
ros2 service call /dr/list_trajs dr_interfaces/srv/ListTrajs {}
ros2 service call /dr/delete_traj dr_interfaces/srv/DeleteTraj {name: 'tarea_1'}
```

- **Envío de un delta de teaching** para verificar el desplazamiento incremental del efector final:

```
ros2 topic pub --once /dr/teaching_delta dr_interfaces/msg/TeachingDelta {dx
: 0.01, dy: 0.0, dz: 0.0, droll: 0.0, dpitch: 0.0, dyaw: 0.0}
```

- **Ejecución de una trayectoria** mediante la acción `/dr/execute`: el goal recibe la trayectoria aplanada `q_traj` y el entero `points`, por lo que desde consola se combina con la carga previa vía `/dr/load_traj`. El feedback reporta el índice del punto en curso y el progreso porcentual:

```
ros2 action send_goal /dr/execute dr_interfaces/action/Execute {q_traj: [...],
points: N} --feedback
```

Este flujo de verificación incremental garantizó que cada nodo cumpliera su contrato de interfaz antes de ser integrado al sistema completo, reduciendo significativamente el tiempo de depuración en las etapas de integración.

Referencias

- [1] Grand View Research. *Global Warehouse Robotics Market Size & Outlook, 2023–2030*. 2024. Disponible en: <https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market-size/global>
- [2] Grand View Research. *Warehouse Robotics Market Outlook: United States*. 2024. Disponible en: <https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market/united-states>
- [3] Grand View Research. *Warehouse Robotics Market Outlook: China*. 2024. Disponible en: <https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market/china>
- [4] Grand View Research. *Warehouse Robotics Market Outlook: Latin America*. 2024. Disponible en: <https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market/latin-america>
- [5] Grand View Research. *Warehouse Robotics Market Outlook: Brazil*. 2024. Disponible en: <https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market/brazil>
- [6] Stuart Russell & Peter Norvig. *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson, 2021.
- [7] John J. Craig. *Introduction to Robotics: Mechanics and Control* (4th ed.). Pearson, 2021.
- [8] Peter Corke. *Robotics, Vision and Control: Fundamental Algorithms in MATLAB* (2nd ed.). Springer, 2023.
- [9] Michael F. Ashby. *Materials Selection in Mechanical Design* (6th ed.). Elsevier, 2022.
- [10] Klaus-Jürgen Bathe. *Finite Element Procedures*. Prentice Hall, 2006.
- [11] Ramu Krishnan. *Permanent Magnet Brushless DC Motor Drives and Controls*. CRC Press, 2017.
- [12] Aaron Martínez & Enrique Fernández. *Learning ROS 2: Basics, Concepts, and Practical Applications*. Packt, 2024.
- [13] Jesse H. Hyman. *Learning Robotics Simulation Using ROS and Gazebo*. Packt, 2018.